述語変換子意味論と余代数

圏論を用いたモデル検査入門

こっとん (@StoneDuality)

2025 年 12 月 20 日 土曜日

本記事は Mathematical Logic Advent Calendar 2025 の 20 日目の投稿です。
このドキュメントは，見やすく読み間違えにくいユニバーサルデザインフォントを採用しています。

目次

0 前置き ・・・

0　前置き

Todo:
1) ループ不変量の例を 4 つくらい追加で挙げる。
2) モナドとクライスリ圏の説明を簡潔に書く。
3) 最終章をちゃんと整理して書く。

0.1　この記事が想定する対象読者

0.2　この記事の目標

この記事では、以下のことを目標とします。

-
-
-

0.3　この記事では扱わないこと

本記事では、以下のことは扱いません。

-
-
-

また、以下の事柄については、既知のものとし、本文中で仮定します。（適宜、勉強して補いながら読んでください。）

- 順序集合や不動点理論の基本事項（ナスター・タルスキの定理、クリーネ / クーゾー・クーゾーの定理、など）
- 数理論理学に関する基本事項（古典命題論理、順序数の基本的な性質、様相論理のクリプキフレーム、など）
- 圏論の基本事項（圏の定義、関手、自然変換、極限と余極限、随伴、カルテシアン閉圏、など）
- 形式言語理論の基本事項（正規言語、有限オートマトン、非決定性オートマトンの決定化、など）

1　導入: モデル検査とは

モデル検査は、大まかに言うと**形式検証**と呼ばれる分野の中に位置づけられます。

- **形式検証** (formal verification):
  ··· ゴールは、「ソフトウェアの**正しさの証明** (correctness proof) を与えること」。
  （発生確率の低い、小さなバグであっても、時として多大な金銭的損害や人命的被害をもたらしうる。）

  1. **定理証明** (theorem proving):
     - **対話型定理証明** (interactive theorem proving): Lean, Rocq/Coq, Agda, など
       ··· 数学的な証明を書くためのソフトウェア。
     - **自動定理証明** (automatic theorem proving): SAT/SMT solvers など
       ··· 形式的な仕様が与えられると、証明や反例を探索し、その性質が成り立つかどうかを判定する。
  2. **モデル検査** (model checking):
     ··· 正しさの問題 (correctness problem) を、"グラフ理論的な問題" に帰着させ、
     "グラフ・アルゴリズム" を使ってその問題を解く。
     （ここでの "グラフ理論的" とは、状態遷移システム (transition system) といった広い意味で。）

モデル検査において、示したいことは、「**モデル** M のもとで**仕様** φ（望ましい性質）が成り立つかどうか」です。

$$M \models \varphi \quad (M \text{ satisfies } \varphi)$$

仕様は、時相論理の論理式（例えば $\mathbf{G}\,(\mathit{req} \to \mathbf{F}\,\mathit{resp})$「リクエストが生じたらレスポンスが返ってくる」など）で書かれることが典型的ですが、ここでは特定の形式化に限定せず、大きく分けて２種類の仕様に注目しましょう。

- 安全性 (safety property): 悪いことが決して起こらない
- 到達性 (reachability property): 良いことがいつか起こる

さまざまな分野

-
-
-
- **オートマトン学習** (automata learning): ブラックボックスシステム（内部の挙動が不明なシステム）に対し、その入出力をもとに、機械学習を用いてそのシステムの挙動を近似的に説明するオートマトンを作って、それを用いてテストを効率的に行う手法。
-

2　ホーア論理と述語変換子 ——— まずは非圏論的な話から

ホーア論理（Hoare logic）と**述語変換子**（predicate transformer）は、プログラムの正しさ（意図した挙動をするか、バグがないかどうか）を数学的に検証するための、互いに密接な、異なる２つのアプローチです。プログラムが満たしていてほしい性質（仕様）を論理式で記述し、それが満たされているかを論理学的手法を用いて確かめます。

たとえば、以下の簡単な例「入力値 N に対し、階乗 $N!$ を計算して出力するプログラム」を考えてみましょう。

```
n := N
k := 1
while (n>0) {
 k := k*n;
 n := n-1;
}
```

例: $N = 4$ の場合

ループ回数	n の値	k の値
はじめ	$N = 4$ (入力)	1
1 回後	3	$4 \times 1 = 4$
2 回後	2	$4 \times 3 = 12$
3 回後	1	$12 \times 2 = 24$
4 回後	0	$24 \times 1 = 24$ (出力)

【このプログラムについて知りたいこと】

- 事後条件「$k = N!$」（出力 k が入力 N の階乗になる）を保証するには、何が分かればよいか？
- 事後条件「$k = N!$」を成り立たせるためには、最低限、どんな事前条件を課せばよいか？

N に正の自然数を入力してこのプログラムを実行した後に、事後条件「$k = N!$」を満たしてほしいものとします。ループが何回実行されるかわからない中で、あらゆる入力 N に対してどうやって保証すればよいでしょうか。答えは、毎回のループを通過する前後で（各変数の値が変わっても）常に保存される性質（**ループ不変量**）「$k \cdot n! = N!$」に注目する、というアイデアです。そうすることで「階乗を出力する」という仕様が必ず満たされることを保証できます。つまり、事後条件を保証するためには、ループ不変量を探せばよいのです。これが**ホーア論理**の考え方です。

では今度は、この事後条件「$k = N!$」を成立させるためには、事前条件として最低限どんな条件を課す必要があるか、という問題を考えてみましょう。答えは、「$N \geq 0$」です。これを知るには、事後条件から出発してプログラムを後ろから前へたどり、各計算に応じて条件を置き換えていくという操作をする必要があります。ただし、ループが何回実行されるかわからないため、この操作の不動点をとります。これが**述語変換子**、とりわけ**最弱事前条件**の考え方です。

2.1　ホーア論理

ホーア論理では $\{P\}\, C\, \{Q\}$ という三つ組を使います。これを**ホーア三つ組**（Hoare triple）と呼びます。これは、「事前条件 P から始めて、プログラム C を実行した結果、事後条件 Q が成り立つ」という関係です。

定義 1　ホーア論理の導出規則は以下のものからなる。

$$\frac{}{\{P\}\ \mathsf{skip}\ \{P\}}\ \text{(skip)}$$

$$\frac{}{\{P\,[a/x]\}\ x := a\ \{P\}}\ \text{(assignment)}$$

$$\frac{\{P\}\, C_1\, \{S\} \qquad \{S\}\, C_2\, \{Q\}}{\{P\}\, C_1; C_2\, \{Q\}}\ \text{(sequential composition)}$$

$$\frac{\{P \wedge \varphi\}\, C_1\, \{Q\} \qquad \{P \wedge \neg\varphi\}\, C_2\, \{Q\}}{\{P\}\ \mathsf{if}\ \varphi\ \mathsf{then}\ C_1\ \mathsf{else}\ C_2\ \{Q\}}\ \text{(if)}$$

$$\frac{\{P \wedge \varphi\}\, C\, \{P\}}{\{P\}\ \mathsf{while}\ \varphi\ \mathsf{do}\ C\ \{P \wedge \neg\varphi\}}\ \text{(while)}$$

$$\frac{P \Rightarrow P' \qquad \{P'\}\, C\, \{Q'\} \qquad Q' \Rightarrow Q}{\{P\}\, C\, \{Q\}}\ \text{(consequence)}$$

例 2　たとえば以下は、ホーア論理の証明図の例である。

$$\dfrac{\dfrac{\begin{array}{c}x{\geq}0{\wedge}x{>}0\\ \Rightarrow x{-}1{\geq}0\end{array} \qquad \dfrac{}{\{x{-}1{\geq}0\}\ x{:=}x{-}1\ \{x{\geq}0\}}\ \text{(a)}}{\{x{\geq}0{\wedge}x{>}0\}\ x{:=}x{-}1\ \{x{\geq}0\}}\ \text{(c)} \qquad \dfrac{\begin{array}{c}x{\geq}0{\wedge}\neg(x{>}0)\\ \Rightarrow x{\geq}0\end{array} \qquad \dfrac{}{\{x{\geq}0\}\ x{:=}x\ \{x{\geq}0\}}\ \text{(a)}}{\{x{\geq}0{\wedge}\neg(x{>}0)\}\ x{:=}x\ \{x{\geq}0\}}\ \text{(c)}}{\{x{\geq}0\}\ \text{if}\ x{>}0\ \text{then}\ x{:=}x{-}1\ \text{else}\ x{:=}x\ \{x{\geq}0\}}\ \text{(i)}$$

注意　ループ不変量とは、以下のような while 文の証明図に現れる I のこと。つまり、

1. ループの入口で事前条件によって含意され（$A \Rightarrow I$）、
2. 各回のループ実行時に常に保たれ（$\{I \wedge \varphi\}\ C\ \{I\}$）、
3. ループの出口で事後条件を含意する（$I \wedge \neg\varphi \Rightarrow B$）

ような論理式 I のことである。

$$\dfrac{A \Rightarrow I \qquad \dfrac{\{I \wedge \varphi\}\ C\ \{I\}}{\{I\}\ \text{while}\ \varphi\ \text{do}\ C\ \{I \wedge \neg\varphi\}}\ \text{(while)} \qquad I \wedge \neg\varphi \Rightarrow B}{\{A\}\ \text{while}\ \varphi\ \text{do}\ C\ \{B\}}\ \text{(conseq)}$$

先ほどの「階乗を計算するプログラム」のホーア論理の証明図を書くと（長くなるので省略するが）I のところに現れる論理式はまさに「$k \cdot n! = N!$」となる。これが、先ほどのプログラムの while 文の「ループ不変量」にあたる。

2.2　述語変換子

述語変換子のひとつである、**最弱事前条件** (weakest precondition) は、述語 Q を別の述語 $wp[\![C]\!](Q)$ に変換します。この述語 $wp[\![C]\!](Q)$ は、「プログラム C について、実行後に事後条件 Q を成立させるような事前条件のうち、論理的含意関係に関して最も弱いもの」です。

定義 3（最弱事前条件意味論）　状態集合を S とする。S 上の述語の集合を $\mathrm{Pred}_S = \{Q \mid Q : S \to \{0,1\}\}$ と定める。なお、各述語 $Q : S \to \{0,1\}$ は、$Q = \{s \in S \mid s \models Q\}$ だと考えればよい。述語どうしの順序は、論理的含意関係によって定めるものとする。そうすると、$(\mathrm{Pred}_S, \sqsubseteq)$ は完備束をなす。

$$P \sqsubseteq Q \quad \text{iff} \quad P \Rightarrow Q$$

プログラム C の最弱事前条件 (weakest precondition) とは、述語変換子（述語を別の述語に変換する写像）

$$wp[\![C]\!] : \mathrm{Pred}_S \to \mathrm{Pred}_S$$

で単調性 $Q_1 \sqsubseteq Q_2 \ \Rightarrow\ wp[\![C]\!](Q_1) \sqsubseteq wp[\![C]\!](Q_2)$ を満たすものである。以下のように帰納的に定義される。

- $wp[\![\text{skip}]\!](Q) = Q$
- $wp[\![x := a]\!](Q) = Q[a/x]$
- $wp[\![C_1; C_2]\!](Q) = wp[\![C_1]\!](wp[\![C_2]\!](Q))$
- $wp[\![\text{if}\ \varphi\ \text{then}\ C_1\ \text{else}\ C_2]\!](Q) = (\varphi \wedge wp[\![C_1]\!](Q)) \vee (\neg\varphi \wedge wp[\![C_2]\!](Q))$
- $wp[\![\text{while}\ \varphi\ \text{do}\ C]\!](Q) = \mu X.\ \underbrace{((\varphi \wedge wp[\![C]\!](X)) \vee (\neg\varphi \wedge Q))}_{\text{loop characteristic function}\ \Phi_Q(X)}$

注意　ここで、μ は順序 $\sqsubseteq$ についての最小不動点をとる操作である。この演算子 $\Phi_Q : \mathrm{Pred}_S \to \mathrm{Pred}_S$ は単調 (monotone) なので、ナスター・タルスキの定理 (the Knaster-Tarski theorem) より、最小不動点が存在する。

上記では、「ホーア三つ組」と「最弱事前条件」の２つが導入されました。ここで、両者を整理しておきましょう。

$$\begin{cases} \{P\}\, C\, \{Q\} & \text{Hoare-style} \\ P \Rightarrow wp[\![C]\!](Q) & \text{Dijkstra-style} \end{cases}$$

両者の関係として、以下が同値になることが知られています。

$$\{P\}\, C\, \{Q\} \quad \text{iff} \quad P \Rightarrow wp[\![C]\!](Q)$$

両者には、以下の違いがあります。

- ホーア三つ組は「関係的」(relational)。
  各述語 P に対し、$\{P\}\, C\, \{Q\}$ が成り立つような述語 Q は複数ありうる。
  各述語 Q に対し、$\{P\}\, C\, \{Q\}$ が成り立つような述語 P は複数ありうる。
- 最弱事前条件は「関数的」(functional)
  各述語 Q に対し、述語 $wp[\![C]\!](Q)$ は一意に定まる。

また、wp から Hoare triple を「再現」することができます。

- $\{\, wp[\![C]\!](Q)\, \}\, C\, \{Q\}$ はいつでも成り立つ。

さて、これら Hoare-style と Dijkstra-style には、それだけでなく、より本質的な違いもあります。それは、部分正当化 (partial correctness) か、全正当性 (total correctness) か、の違いです。次節ではその点について考えます。

2.3 部分正当性と全正当性

プログラムの正しさを検証するうえで、**部分正当性** (partial correctness) と**全正当性** (total correctness) という２種類の「正しさ」の考え方があります。ホーア三つ組 $\{P\}\, C\, \{Q\}$ は前者（部分正当性）を主張するものです。ここでさしあたり、後者（全正当性）のほうを $[P]\, C\, [Q]$ と表記して区別することにし、両者の違いを比べてみましょう。

- **部分正当性** (partial correctness) $\{P\}\, C\, \{Q\}$
  - 「事前条件 P から始めて、**もし**プログラム C が**停止したら**、その実行結果は事後条件 Q を満たす。」
  - プログラムが停止しないときには何も保証しない。
- **全正当性** (total correctness) $[P]\, C\, [Q]$
  - 「事前条件 P から始めて、プログラム C が**必ず停止し**、**かつ**、その実行結果は事後条件 Q を満たす。」
  - プログラムの停止性 (termination) も保証する。

ホーア三つ組が保証するのは部分正当性 (partial correctness) なのに対し、最弱事前条件が保証するのは全正当性 (total correctness) です。この違いは、while ループの不動点のとり方に端を発しています。

- 述語変換子における while 文の解釈は、最小不動点で与えられていたことを、思い出しましょう。

$$wp[\![\text{while } \varphi \text{ do } C]\!](Q) = \mu X.\ \underbrace{((\varphi \wedge wp[\![C]\!](X)) \vee (\neg\varphi \wedge Q))}_{\text{loop characteristic function } \Phi_Q(X)}$$

- それに対し、ホーア論理における while 文の解釈は、実は、最大不動点によって与えられていたのです。

$$\dfrac{P \Rightarrow I \qquad \dfrac{\{I \wedge \varphi\}\, C\, \{I\}}{\{I\}\, \text{while } \varphi \text{ do } C\, \{I \wedge \neg\varphi\}}\,(\text{while}) \qquad I \wedge \neg\varphi \Rightarrow Q}{\{P\}\, \text{while } \varphi \text{ do } C\, \{Q\}}\,(\text{conseq})$$

命題 4 $(L, \sqsubseteq)$ を完備束とし、$f : L \to L$ を単調写像とする。このとき、以下の帰結関係（上段から下段）が成り立つ。

$$\frac{p \sqsubseteq i \qquad i \sqsubseteq f(i) \qquad i \sqsubseteq q}{p \sqsubseteq \nu(f(-) \sqcap q)}$$

証明 $\sqcap$ の普遍性より、以下の同値関係が成り立つ。

$$\frac{i \sqsubseteq f(i) \qquad i \sqsubseteq q}{i \sqsubseteq f(i) \sqcap q}$$

ナスター・タルスキの定理より、

$$\nu(f(-) \sqcap q) = \bigsqcup \{x \in L \mid x \sqsubseteq f(x) \sqcap q\}$$

であるから、特に任意の $i \in L$ について以下の帰結関係が成り立つ。

$$\frac{i \sqsubseteq f(i) \sqcap q}{i \sqsubseteq \nu(f(-) \sqcap q)}$$

ここで、条件 $p \sqsubseteq i$ と推移性から、最終的に

$$\frac{p \sqsubseteq i \qquad i \sqsubseteq f(i) \qquad i \sqsubseteq q}{p \sqsubseteq \nu(f(-) \sqcap q)}$$

が得られた。 □

上記の命題の中の、p は事前条件に、q は事後条件に、i はループ不変量に、それぞれ相当しています。

$$\frac{p \sqsubseteq i \qquad i \sqsubseteq f(i) \qquad i \sqsubseteq q}{p \sqsubseteq \nu(f(-) \sqcap q)} \quad \rightsquigarrow \quad \frac{P \Rightarrow I \qquad \dfrac{\{I \wedge \varphi\}\, C\, \{I\}}{\{I\}\ \mathsf{while}\ \varphi\ \mathsf{do}\ C\ \{I \wedge \neg\varphi\}} \qquad I \wedge \neg\varphi \Rightarrow Q}{\{P\}\ \mathsf{while}\ \varphi\ \mathsf{do}\ C\ \{Q\}}$$

ホーア論理では最大不動点 (gfp) ν を、最弱事前条件では最小不動点 (lfp) μ をとっていたことがわかります。

なお、述語変換子の中にも、部分正当化をおこなうものもあり、それは「**最弱リベラル事前条件**」(weakest liberal precondition, wlp) と呼ばれます。これは、while ループの解釈に lfp ではなく gfp を用いる点だけが異なります。

- $wp[\![\mathsf{while}\ \varphi\ \mathsf{do}\ C]\!](Q) = \mu X.\, ((\varphi \wedge wp[\![C]\!](X)) \vee (\neg\varphi \wedge Q))$
- $wlp[\![\mathsf{while}\ \varphi\ \mathsf{do}\ C]\!](Q) = \nu X.\, ((\varphi \wedge wlp[\![C]\!](X)) \vee (\neg\varphi \wedge Q))$

以上を表に整理すると、以下の通りです。

	Hoare logic	predicate transformer
gfp (最大不動点)	partial correctness $\{P\}\, C\, \{Q\}$ 「P から始めて、もし停止するなら Q」	weakest liberal precondition $wlp[\![C]\!](Q)$ 「もし停止するなら Q を満たす最弱条件」
lfp (最小不動点)	total correctness $[P]\, C\, [Q]$ 「P から始めて、必ず停止してかつ Q」	weakest precondition $wp[\![C]\!](Q)$ 「必ず停止してかつ Q を満たす最弱条件」

直観: 一般に、順序集合における最小不動点と最大不動点の間には $\mu x.\, \Phi x \sqsubseteq \nu x.\, \Phi x$ という大小関係があるわけですが、このギャップが「プログラムの停止性を保証するかしないか」の違いだ、と理解するのがわかりやすいでしょう。

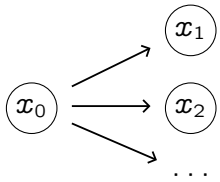
さて、ここまで抽象的な話が続いたので、次節でいったん具体例を見ておきましょう。

2.4 述語変換子を用いたモデル検査の簡単な例

本節では、述語変換子（前節で導入した最弱事前条件に限らず一般の述語変換子）を使って実際にモデル検査をする方法を、簡単な例を挙げて紹介します。以下では例として、クリプキフレーム $(X, \delta : X \to \mathcal{P}(X))$ を考えます。X は状態集合で、δ は遷移関数です。（これは、次章で導入する用語で言うと「余代数」なのですが、詳しくは後で述べます。）

$$\begin{array}{rccc} \delta : & X & \longrightarrow & \mathcal{P}(X) \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & x & \longmapsto & \delta(x) = \{x' \mid x \to x'\} \end{array}$$

絵で描くとたとえば次のようなものです。



L を完備束とし、以下で定められているものとします。

$$L := 2^X = \{0, 1\}^X$$

この L は述語の集合であり（補足: $p(x) = 1$ を $x \models p$ と考えます）、L 上の順序 $\leq$ は述語どうしの論理的含意関係によって定められているものとします。また、L の最小元と最大元をそれぞれ $\bot, \top$ と書くことにします。

ここで、述語変換子 $f : 2^X \to 2^X$ を、次のように定めます。（補足: この f は単調 (monotone) な関数です。）

$$\begin{array}{rccl} f : & 2^X & \longrightarrow & 2^X \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & p & \longmapsto & \{x \mid \forall x' \ (x' \in \delta(x) \Rightarrow x' \in p)\} \\ & & & = \{x \mid \text{all successors of } x \text{ satisfy } p\}. \end{array}$$

そして、この f の最小不動点 μf をとります。（補足: この μf は、L の要素、すなわち述語です。）

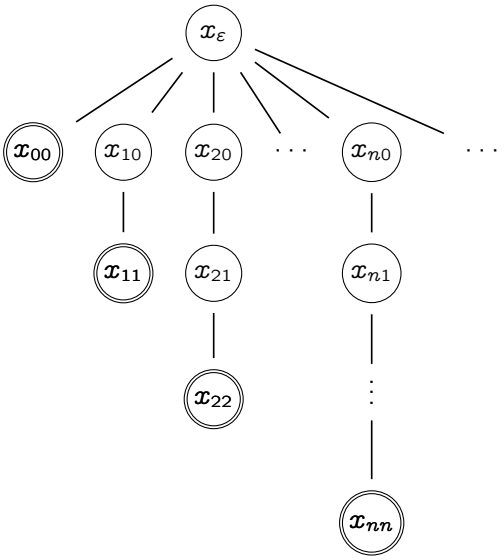
$$\mu f = \{x \mid \text{every path from } x \text{ reaches a state with no successors}\}$$

μf は、以下の図に見られるような 2^X における鎖の、超限反復 (transfinite iteration) によって構成されます。（最小不動点 μf の近似列は、下から積み上げます。）

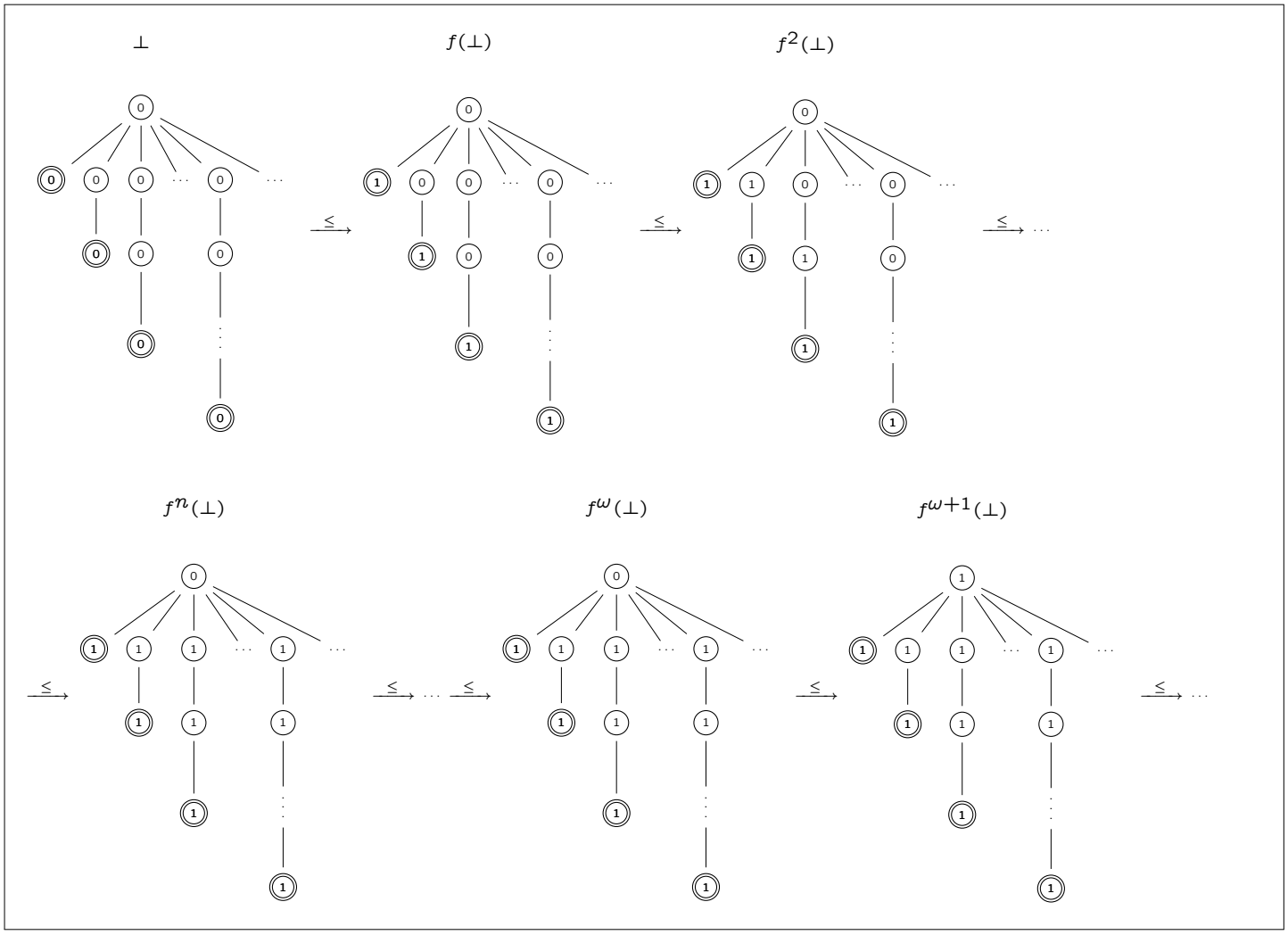
$$\begin{tikzcd}
\bot \arrow[r, "\leq"] \arrow[dd, equal] \arrow[rrrrrr, bend left] & f(\bot) \arrow[r, "\leq"] \arrow[dd, equal] \arrow[rrrrr, bend left] & f^2(\bot) \arrow[r, "\leq"] \arrow[dd, equal] \arrow[rrrr, bend left] & \cdots \arrow[r, "\leq"] & f^{\omega}(\bot) \arrow[r, "\leq"] \arrow[dd, equal] & f^{\omega+1}(\bot) \arrow[r, "\leq"] & \cdots \\
\emptyset & \{x \mid \delta(x) = \emptyset\} & \begin{array}{l}\{x \mid \text{all paths are} \\ \quad \text{of length} \leq 2\}\end{array} & & \sup\limits_{\beta<\omega} f^{\beta}(\bot) & &
\end{tikzcd}$$

（補足: この f は単調だが、スコット連続であるとは限らないため、ω よりも先まで（安定 (stabilize) するまで）反復を回し続ける必要があります。）

ここで具体的に、以下のクリプキフレーム (X, δ) について、上で定めた述語 μf を実際に検証してみましょう。



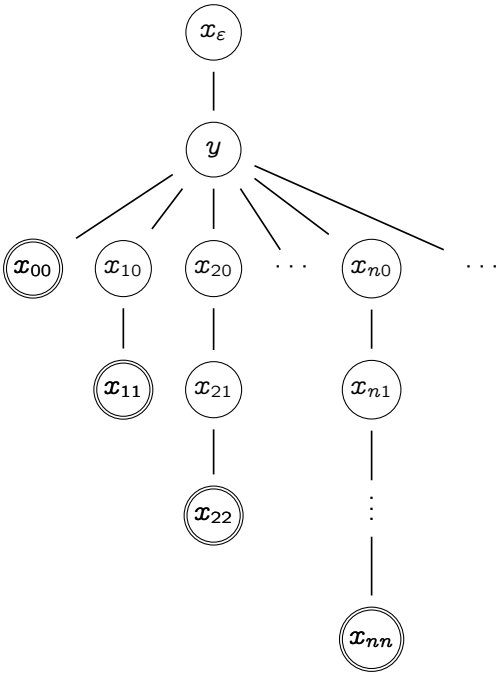
このクリプキフレームにおいて $x_\varepsilon \models \mu f$ が成り立つことが、以下の図のような超限反復からわかります。



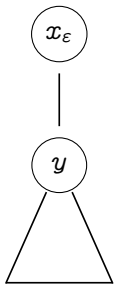
この鎖は、$\omega + 1$ で安定 (stabilize) し、そこから先は変わらない、つまり $f^{\omega+1}(\bot)$ が最小不動点となります。

補足: $f^\omega(\bot)$ は、これより左に現れるすべての上限 (supremum) をとったものです。(てっぺん x_ε にはまだ 1 が現れたことがないので、それらの上限は 0 のままです。) それに対し、次の $f^{\omega+1}(\bot)$ でやっと、てっぺん x_ε にも 1 が現れます。

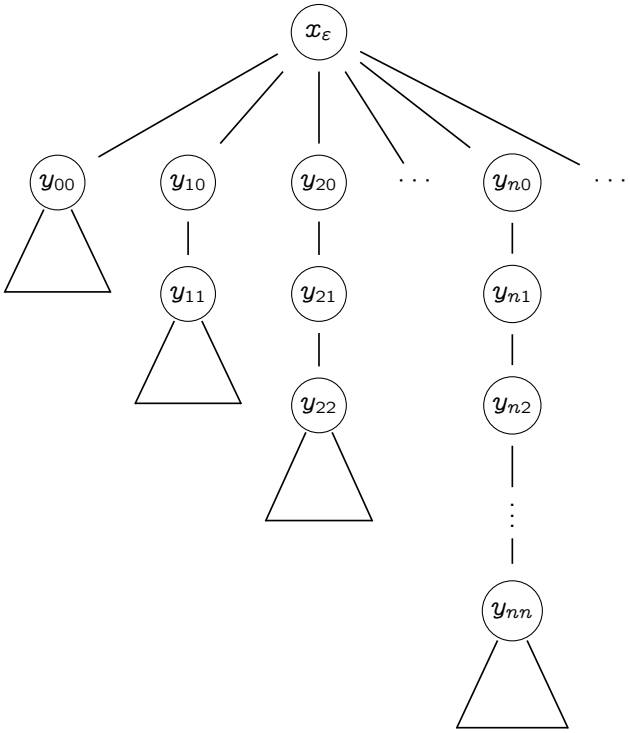
$\omega + 1$ よりもさらに先で安定（stabilize）する例もあります。分岐がもっと複雑なものを考えればよいです。たとえば、以下のようなクリプキフレームを考えてみましょう。この場合、$\omega + 2$ で安定します。



（以降、左の図を以下のように略記することにします。）



さらに、もっと複雑なものを考えてみます。たとえば、（上記の略記を用いて）以下の図のクリプキフレームを考えましょう。この場合、$\omega \cdot 2 + 1$ で安定します。



さて、本章でここまで扱ってきた述語変換子の考え方は、圏論の言葉を使うととても見通しがよくなります。そのための準備として、次章では、圏論の関手を使ってプログラムの振舞いを捉える方法について紹介します。（すでに「余代数」や「モナド」や「グロタンディーク・ファイブレーション」のことは知っているよ、という人は 4 章へ飛んでください。）

3　圏論の関手でシステムの振舞いを捉える

情報科学における「**圏論的な手法の強み**」は、抽象的・俯瞰的であるがゆえに、様々な種類の情報システムに対して、既存の議論や知見を再編することで新たな問題設定に柔軟に適応できる点にあると言えます。

本章で紹介する**余代数** (coalgebra) は、そのように多様な状態遷移システム（例: 二分木グラフ、ストリーム、クリプキフレーム、オートマトン、マルコフ連鎖、マルコフ決定過程、etc.）を扱う際にとても役立つ道具です。

3.1　余代数の定義

まずは余代数の定義を述べます。

定義 5（余代数）　$\mathbb{C}$ を圏とし、$F : \mathbb{C} \to \mathbb{C}$ を関手とする。F-**余代数**とは、$\mathbb{C}$ の対象 X と射 $c : X \to F(X)$ の組 (X, c) のことと定める。また、2 つの余代数 $(X,\ c : X \to F(X))$, $(Y,\ d : Y \to F(Y))$ の間の**余代数準同型**とは、射 $f : X \to Y$ であって $d \circ f = F(f) \circ c$ を満たすものとする。

$$\begin{tikzcd} F(X) \arrow[r, "F(f)"] & F(Y) \\ X \arrow[u, "c"] \arrow[r, "f"'] \arrow[ur, phantom, "\circlearrowleft"] & Y \arrow[u, "d"'] \end{tikzcd}$$

定義 6（余代数の圏）　F-余代数の圏を $\mathrm{Coalg}(F)$ と書くことにする。対象が余代数、射が余代数準同型である。

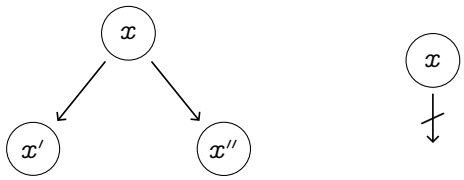
$$\begin{pmatrix} F(X) \\ \uparrow c \\ X \end{pmatrix} \xrightarrow{f} \begin{pmatrix} F(Y) \\ \uparrow d \\ Y \end{pmatrix}$$

3.2　余代数の様々な例

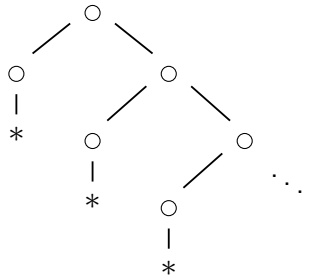
いくつか例を挙げます。

例 7　二分木グラフは、**Set** 上の関手 $F(X) = (X \times X) + 1$ に対する余代数 $c : X \to F(X)$ として与えられる。

分岐　$c(x) = (x', x'')$　　or　　停止　$c(x) = *$

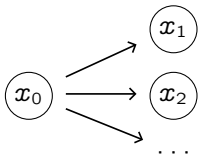


たとえば以下のような二分木グラフが挙げられる。



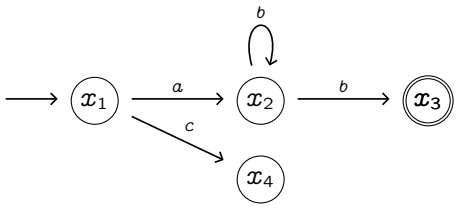
例 8　様相論理におけるクリプキフレーム (Kripke frame) は、余代数 $\delta : X \to \mathcal{P}(X)$ で与えられる。

$$\begin{array}{rccc} \delta : & X & \longrightarrow & \mathcal{P}(X) \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & x & \longmapsto & \delta(x) = \{x' \mid x \to x'\} \end{array}$$



例 9　非決定性オートマトンは、余代数 $\gamma : X \to \mathcal{P}(A \times X) + 1$ で与えられる。なお $1 = \{*\}$ とする。

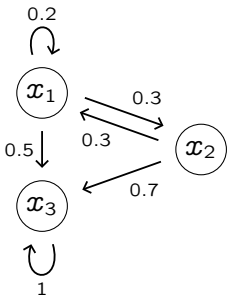
遷移　$\gamma(x) = \{(a, x'), (b, x''), \ldots\}$　　or　　停止　$\gamma(x) = *$



例 10　マルコフ連鎖は、余代数 $\gamma : X \to \mathcal{D}(X)$ で与えられる。なお、関手 $\mathcal{D} : \mathbf{Set} \to \mathbf{Set}$ は

$$\begin{aligned} \mathcal{D}(X) &= \{d : X \to [0,1] \mid \textstyle\sum d(x) = 1\} \\ &= \{\textstyle\sum p_i|x_i\rangle \text{ , where each } p_i \text{ is a probability } d(x_i) \in [0,1].\} \\ \mathcal{D}(f) &: \mathcal{D}(X) \to \mathcal{D}(Y) \\ \mathcal{D}(f)&(\textstyle\sum p_i|x_i\rangle) = \textstyle\sum p_i|f(x_i)\rangle \end{aligned}$$

と定められているものとする。



$$\begin{aligned} \gamma(x_1) &= 0.2|x_1\rangle + 0.3|x_2\rangle + 0.5|x_3\rangle, \\ \gamma(x_2) &= 0.3|x_1\rangle + 0.7|x_3\rangle, \\ \gamma(x_3) &= |x_3\rangle. \end{aligned}$$

注意（分布関手のためのケット記法）　$\mathcal{D}(X) = \{\, d : X \to [0,1] \mid \sum d(x) = 1 \,\}$ の各要素を $\sum p_i|x_i\rangle$ と書く。ここで、各 p_i は確率 $d(x_i) \in [0,1]$ である。例えば、コイントス $X = \{H, T\}$ を考えるとき、「公平なコイン」の分布は $d = \frac{1}{2}|H\rangle + \frac{1}{2}|T\rangle$ と書ける。なお、$|H\rangle = \left(\begin{smallmatrix}1\\0\end{smallmatrix}\right)$ および $|T\rangle = \left(\begin{smallmatrix}0\\1\end{smallmatrix}\right)$ である。

3.3 終余代数とその構成

ここでは、終余代数の定義と、その構成のしかたを紹介します。まずは定義です。

定義 11（終余代数） 終 F-余代数 $\zeta : Z \to F(Z)$ とは、余代数の圏 $\mathrm{Coalg}(F)$ の中の終対象である。つまり、任意の余代数 $c : X \to F(X)$ に対し、一意な余代数準同型 $f : X \to Z$ が存在することである。

$$\begin{tikzcd}
F(X) \arrow[r, dashed, "F(f)"] & F(Z) \\
X \arrow[u, "c"] \arrow[r, dashed, "\exists! f"'] \arrow[ur, phantom, "\searrow"] & Z \arrow[u, "\zeta"']
\end{tikzcd}$$

命題 12（ランベックの補題） $\zeta : Z \to F(Z)$ が終余代数なら、ζ は同型射である。つまり $Z \cong F(Z)$ である。

証明 余代数 $F(\zeta) : F(Z) \to F(F(Z))$ を考える。Z の終対象性より、一意な $g : F(Z) \to Z$ が手に入る。

$$\begin{tikzcd}
F(F(Z)) \arrow[r, dashed, "F(g)"] & F(Z) \\
F(Z) \arrow[u, "F(\zeta)"] \arrow[r, dashed, "g"'] \arrow[ur, phantom, "\searrow"] & Z \arrow[u, "\zeta"']
\end{tikzcd}$$

このとき、$g = \zeta^{-1}$ であることを示す。

1) 第一に、$g \circ \zeta = \mathrm{id}_Z$ である。（Z は終対象のため、$g \circ \zeta : Z \to Z$ と $\mathrm{id}_Z : Z \to Z$ は同一の射。）

$$\begin{tikzcd}
F(Z) \arrow[r, "F(\zeta)"] & F(F(Z)) \arrow[r, "F(g)"] & F(Z) \\
Z \arrow[u, "\zeta"] \arrow[r, "\zeta"'] \arrow[rr, bend right, "\mathrm{id}_Z"'] \arrow[ur, phantom, "\searrow"] & F(Z) \arrow[u, "F(\zeta)"] \arrow[r, "g"'] \arrow[ur, phantom, "\searrow"] & Z \arrow[u, "\zeta"']
\end{tikzcd}$$

2) 第二に、$\zeta \circ g = \mathrm{id}_{F(Z)}$ である。（以下の簡単な計算により示せる。）

$$\begin{aligned}
\zeta \circ g &= F(g) \circ F(\zeta) && \text{(by definition of } g\text{)} \\
&= F(g \circ \zeta) && \text{(functionality of } F\text{)} \\
&= F(\mathrm{id}_Z) && (g \circ \zeta = \mathrm{id}_Z \text{ as we proved}) \\
&= \mathrm{id}_{F(Z)} && \text{(because } F \text{ is a functor)}
\end{aligned}$$

それゆえ、$f \circ \zeta = \mathrm{id}_Z$ と $\zeta \circ f = \mathrm{id}_{F(Z)}$ である。よって、$g = \zeta^{-1}$ である。 □

系 13 べき集合の関手 $\mathcal{P} : \mathbf{Set} \to \mathbf{Set}$ には、終余代数は存在しない。

証明 もし終余代数 $\zeta : X \to \mathcal{P}(X)$ があるとすると、ζ は同型射になるはずである（ランベックの補題）。しかし、カントールの冪集合の定理より、$X \ncong \mathcal{P}(X)$ である。というのも、任意の集合 X について、濃度は $|X| < |\mathcal{P}(X)|$ であるからである。以上より、**Set** において終 $\mathcal{P}$-余代数は存在しない。 □

ここからは、どうやって終余代数を構成するかについて見ていきます。

命題 14 …

証明 … □

証明を書く。ordinal な議論に気をつけて。

補足: 終 F-余代数の carrier は、関手 F の最大不動点にあたるものであるため、それを νF と書くことが多いです。すなわち、終 F-余代数とは $\zeta : \nu F \xrightarrow{\cong} F(\nu F)$ という同型射です。

3.4 代数と始代数、余代数と終余代数 ——— 異なる 2 種類の “無限” をとらえる

さて、F-余代数の圏論的双対である、F-代数を導入します。

定義 15（代数） …

定義 16（始代数） …

命題 17（ランベックの補題） …

命題 18（始代数の構成） …

主張だけ書く

補足: 始 F-代数の carrier は、関手 F の最小不動点にあたるものであるため、それを μF と書くことが多いです。すなわち、始 F-代数とは $\zeta : F(\mu F) \xrightarrow{\cong} \mu F$ という同型射です。

ではここで、**Set** 上の多項式関手（集合演算 $+, \times, (-)^n$ だけを用いて書けるような関手）について、始代数と終余代数を具体的に求める練習をしましょう。

考え方

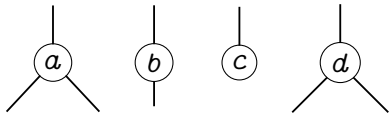
まず、**Set** 上の関手 $F_\Sigma : \mathbf{Set} \to \mathbf{Set}$ が**多項式関手** (polynomial functor) であるとは、

$$F_\Sigma \;=\; \sum_{\sigma \in \Sigma} (-)^{|\sigma|}$$

の形で表される関手のことです。ここで、Σ は**ランクつきアルファベット** (ranked alphabet) とします。

たとえば、$\Sigma = \{a : 2,\ b : 1,\ c : 0,\ d : 2\}$ のとき、 $F_\Sigma = \underbrace{(-)^2}_{a} + \underbrace{(-)^1}_{b} + \underbrace{1}_{c} + \underbrace{(-)^2}_{d}$ です。

このとき、**Σ-木** (Σ-tree) とは、以下の図のような node で構成される木のことです。



ここで、μF（始代数の carrier set）と νF（終余代数の carrier set）は以下の通りになります。

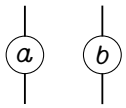
$$\begin{aligned} \mu F &= \{\text{ all well-founded } \Sigma\text{-trees } \} \\ \nu F &= \{\text{ all } \Sigma\text{-trees, possibly with infinite paths } \} \end{aligned}$$

例 1 例として、$F = A \times (-)$ という polynomial functor を考えてみましょう。

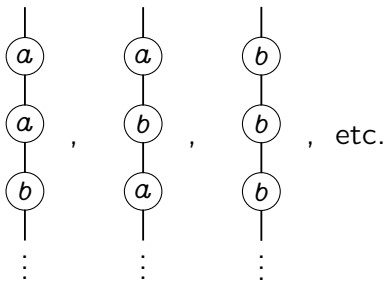
$$\begin{aligned} F &= A \times (-) \\ &= \underbrace{(-)^1 + \cdots + (-)^1}_{|A|} \end{aligned}$$

いま、簡単のため $A = \{a, b\}$ としましょう。（ここで、a, b は rank が 1 の文字です。）

$$\begin{aligned} F &= \{a, b\} \times (-) \\ &= \underbrace{(-)^1}_{a} + \underbrace{(-)^1}_{b} \end{aligned}$$



ここで、各 A-tree は、以下の図のような、無限に続く一本木となります。



このとき、μF と νF は次のようになります。（補足:「有限長で止まる木」はないので、μF は空集合です。）

$$\mu F = \emptyset$$

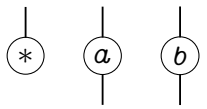
$$\nu F = A^\omega$$

例 2 次に、$F = 1 + A \times (-)$ を考えてみましょう。

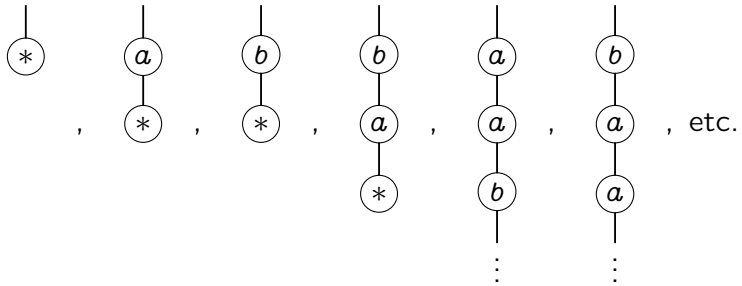
$$\begin{aligned} F &= 1 + A \times (-) \\ &= (-)^0 + \underbrace{(-)^1 + \cdots + (-)^1}_{|A|} \end{aligned}$$

いま、$1 = \{*\}$, $A = \{a, b\}$ としましょう。つまり $\Sigma = \{* : 0,\ a : 1,\ b : 1\}$ です。

$$\begin{aligned} F &= 1 + \{a, b\} \times (-) \\ &= \underbrace{(-)^0}_{*} + \underbrace{(-)^1}_{a} + \underbrace{(-)^1}_{b} \end{aligned}$$



ここで、各 Σ-tree は、以下のような、有限で止まるか、無限に続く、一本木となります。



このとき、μF と νF は次のようになります。（有限で止まるものが A^*、無限に続きうるものを含めて A^∞ です。）

$$\mu F = A^*$$

$$\nu F = A^\infty \ (= A^* + A^\omega)$$

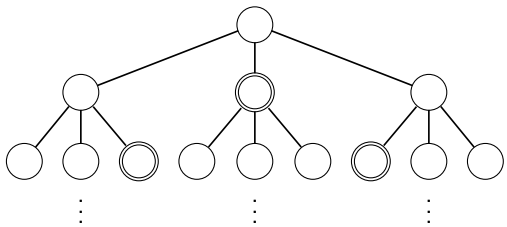
例 3 さらに、$F = 2 \times (-)^A$ も考えてみましょう。A は有限な非空集合とします。
この F-余代数は、決定性オートマトンです。

$$\dfrac{\dfrac{X \longrightarrow 2 \times X^A}{X \longrightarrow 2} \qquad \dfrac{X \longrightarrow X^A}{A \times X \longrightarrow X}}{}$$

たとえば $|A| = 3$ のとき、

$$\begin{aligned} F &= 2 \times (-)^3 \\ &= \underbrace{(-)^3}_{\bigcirc} + \underbrace{(-)^3}_{\circledcirc} \end{aligned}$$

となります。ここで、各 A-tree は、$|A|$-branching tree with 2-colors となります。（下の図は $|A| = 3$ の場合。）
以下の図のような、色のつきかた（$\bigcirc$ か $\circledcirc$ か）だけが違う、形の同じ、高さ無限の木が、たくさん作れます。



このとき、νF は、そのような木たちの全体なので、A 上の言語全体、つまり 2^{A^*} となります。
（補足: 各言語 $L \subseteq A^*$ は、「各語 $w \in A^*$ を受理するかどうか」を 0 か 1 で返す関数 $L : A^* \to 2$ です。）

$$\mu F = \emptyset$$

$$\nu F = 2^{A^*} \ (\cong \mathcal{P}(A^*))$$

まとめると

	μF carrier of initial algebra	νF carrier of final coalgebra
$F = A \times (-)$	$\emptyset$	A^ω
$F = 1 + A \times (-)$	A^*	A^∞ $(= A^* + A^\omega)$
$F = 2 \times (-)^A$	$\emptyset$	2^{A^*} $(\cong \mathcal{P}(A^*))$
$F = B \times (-)^A$	$\emptyset$	B^{A^*}
$F = C + B \times (-)^A$	finite $\lvert A\rvert$-ary trees with B-labeled nodes and C-labeled leaves	possibly infinite $\lvert A\rvert$-ary trees with B-labeled nodes and C-labeled leaves

となります。

ここで「**可能無限**」(potential infinity) と「**実無限**」(actual infinity) の考え方・違いをおさえておきましょう。

- 始代数 (μF) は、有限回の操作で到達できるような無限（＝**可能無限**）を表すことができるのに対し、
- 終余代数 (νF) は、無限に続く振舞いや無限対象（＝**実無限**）までも含めて扱うことができます。

観察 1 一般に $\mu F \subseteq \nu F$ となりますが、この包含関係は、圏論的な観点からは次のように理解されます。

$$\begin{tikzcd}
F(\mu F) \arrow[r, dashed] \arrow[d, shift left, "\cong"] & F(\nu F) \\
\mu F \arrow[u, shift left, "\alpha^{-1}"] \arrow[r, dashed, "\text{mono}"'] & \nu F \arrow[u, "\cong"']
\end{tikzcd}$$

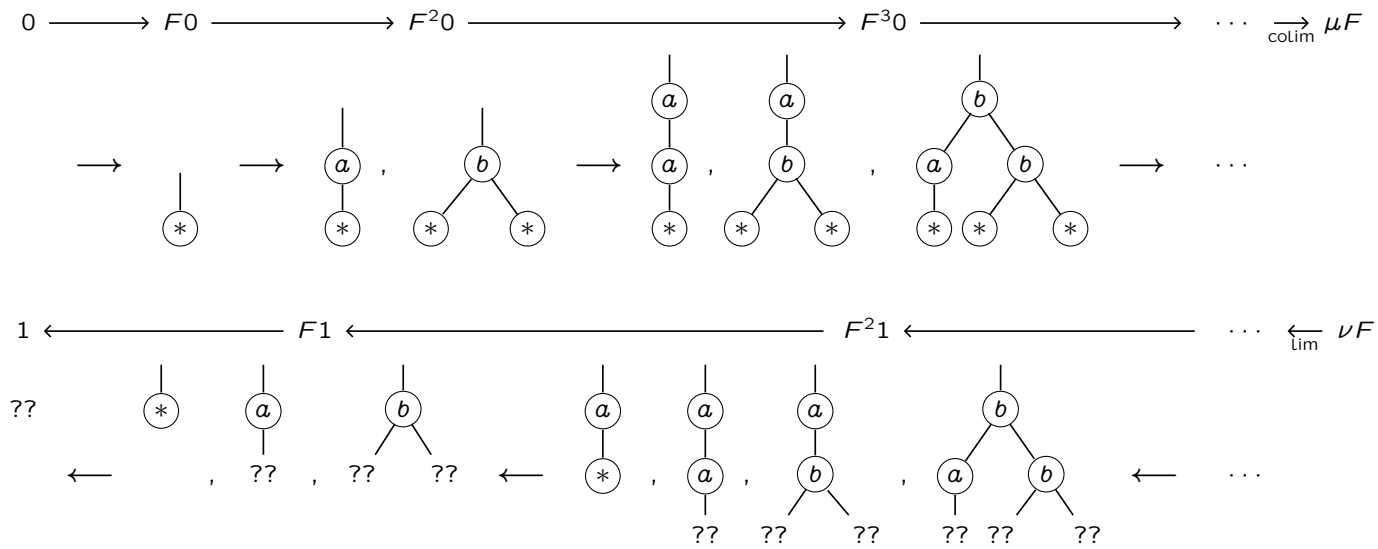
これが mono 射になることは、$F : \mathbf{Set} \to \mathbf{Set}$ が有限交叉を保つ（$F(A \cap B) \cong F(A) \cap F(B)$ for all $A, B \subseteq X$）という条件のもとで成り立ちます。（なお、**Set** 上の多項式関手なら、有限交叉は保たれます。）ここでは詳細は省略。

観察 2 μF は「ボトムアップ」に木が育ち、νF は「トップダウン」に木が育ちます。

例: $F = \underbrace{1}_{*} + \underbrace{(-)^1}_{a} + \underbrace{(-)^2}_{b}$ の場合だと、initial chain と final chain は、それぞれ以下のようになります。

$$0 \longrightarrow F0 \longrightarrow F^2 0 \longrightarrow \cdots \xrightarrow[\text{colim}]{} \mu F$$

$$1 \longleftarrow F1 \longleftarrow F^2 1 \longleftarrow \cdots \xleftarrow[\text{lim}]{} \nu F$$



3.5 計算効果 (computational effect) について

3.2 節でいくつか例を挙げながら紹介したように、余代数 (coalgebra) は「1 ステップの状態遷移」を表します。

$$c : X \to F(X)$$

より一般に、計算 (computation) は、以下のように表されます。(なお $X = Y$ とすれば余代数です。)

$$f : X \to F(Y)$$

ここで「**ただの関数**」(pure function) と「**効果つき計算**」(effectful computation) を区別することは重要です。

pure function	vs	effectful computation
$f : X \to Y$		$f : X \to F(Y)$

典型例として、例外処理 (exception handling) と呼ばれる計算効果を表す関手 $F(Y) = Y + E$ を用いた、効果つきの計算 $f : X \to Y + E$ が挙げられます。ここでは $+$ は余積を表し、E は "エラーの集合" とみなしましょう。このとき、計算 $f : X \to Y + E$ は、単に入力値 $x \in X$ から出力値 $y \in Y$ を返すだけでなく、「もし例外が生じたらエラー $e \in E$ を返す」という**副作用** (side effect) を持ちます。

$$\begin{array}{rccc} f: & X & \longrightarrow & Y + E \\ & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\ & x & \longmapsto & y \\ & x' & \longmapsto & \qquad e \end{array}$$

では、この例のような副作用を持つ計算（効果つき計算）どうしの「合成」はどう考えればいいでしょうか。たとえば、2 つの効果つき計算 $f : X \to Y + E$ と $g : Y \to Z + E$ を合成して、$g \circ f : X \to Z + E$ のようなものを作りたいとき、f, g の両者は型が合っていないため、一筋縄ではいかないことがわかります。そこで、次節で導入する「モナド」や「クライスリ圏」といった道具が役立ちます。

3.6 モナド、クライスリ圏、EM 圏

通常、プログラム意味論において、副作用（非決定性や確率など）を伴う計算は、モナド F を用いた写像 $c : X \to FY$ として表されます。これは、クライスリ圏 $\mathrm{Kl}(F)$ における射 $c : X \nrightarrow Y$ とみなすことができます。

$$\frac{c : X \to FY \quad \text{in the category } \mathbb{C}}{c : X \nrightarrow Y \quad \text{in the Kleisli category } \mathrm{Kl}(F)}$$

以下では、「**モナド**」や「**クライスリ圏**」そして「**アイレンバーグ・ムーア圏**」を導入します。

なお、この節で扱う内容は、Haskell などのプログラム言語にも用いられています。モナドやクライスリ圏などの考え方は、「計算効果」を関数型言語の中で扱うための仕組みとして知られています。(ちなみに、Haskel におけるモナドなどに関する話題についてはこの記事では触れませんが、気になる人はぜひこの機会に自ら勉強してみてください。)

定義 19（モナド）　**モナド** (monad) とは 3 つ組 (T, η, μ) であって、以下を満たすものである。
1) $T : \mathbb{C} \to \mathbb{C}$ は関手である。2) 単位 $\eta : \mathrm{id}_{\mathbb{C}} \Rightarrow T$ は自然変換である。3) 乗法 $\mu : T \circ T \Rightarrow T$ は自然変換である。そして、以下の図式を可換にする。

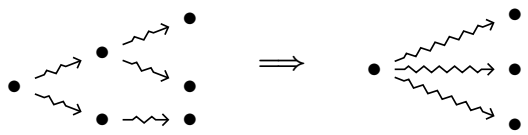
$$\begin{tikzcd}
\mathrm{id}_{\mathbb{C}} \circ T \arrow[r, Rightarrow, "\eta \cdot \mathrm{id}_T"] \arrow[dr, equal] & T \circ T \arrow[d, Rightarrow, "\mu"] & T \circ \mathrm{id}_{\mathbb{C}} \arrow[l, Rightarrow, "\mathrm{id}_T \cdot \eta"'] \arrow[dl, equal] \\
& T &
\end{tikzcd}
\qquad
\begin{tikzcd}
T \circ T \circ T \arrow[r, Rightarrow, "\mathrm{id}_T \cdot \mu"] \arrow[d, Rightarrow, "\mu \cdot \mathrm{id}_T"'] & T \circ T \arrow[d, Rightarrow, "\mu"] \\
T \circ T \arrow[r, Rightarrow, "\mu"'] & T
\end{tikzcd}$$

言い換えると、任意の対象 $X \in \mathbb{C}$ について、$\eta_X : X \to T(X)$ と $\mu_X : T(T(X)) \to T(X)$ は以下を満たす。

$$\begin{tikzcd}
TX \arrow[r, "\eta_{TX}"] \arrow[dr, "\mathrm{id}_{TX}"'] & TTX \arrow[d, "\mu_X"] & TX \arrow[l, "T\eta_X"'] \arrow[dl, "\mathrm{id}_{TX}"] \\
& TX &
\end{tikzcd}
\qquad
\begin{tikzcd}
TTTX \arrow[r, "T\mu_X"] \arrow[d, "\mu_{TX}"'] & TTX \arrow[d, "\mu_X"] \\
TTX \arrow[r, "\mu_X"'] & TX
\end{tikzcd}$$

モナドの例として、**冪集合モナド** (powerset monad) は以下のようなものです。

- $\mathcal{P} : \mathbf{Set} \to \mathbf{Set}$, such that $\mathcal{P}(X) = \{A \mid A \subseteq X\}$ is a powerset of X
  with $\mathcal{P}(f) : \mathcal{P}(X) \to \mathcal{P}(Y)$, such that $\mathcal{P}(f)(A) = \{f(x) \mid x \in A\} \subseteq Y$, for each morphism $f : X \to Y$
- $\eta_X : X \to \mathcal{P}(X)$, no-branching, such that $\eta_X(x) = \{x\}$
- $\mu_X : \mathcal{P}(P(X)) \to \mathcal{P}(X)$, flattening nested branches, such that $\{X_i \subseteq X \mid i \in I\} \mapsto \bigcup_{i \in I} X_i$
  例えば $\{\{x, y\}, \{z\}\} \mapsto \{x, y, z\}$ となり、2 段階の遷移を 1 段階に潰します。



また、**劣分布モナド** (sub-distribution monad) は以下のようなものです。

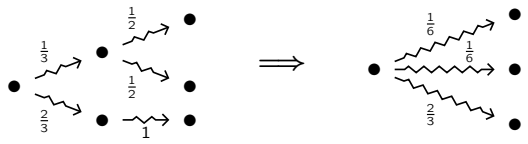
- $\mathcal{D} : \mathbf{Set} \to \mathbf{Set}$, such that

$$\begin{aligned}
\mathcal{D}(X) &= \{d : X \to [0, 1] \mid \textstyle\sum d(x) \leq 1\} \\
&= \{\textstyle\sum p_i |x_i\rangle \text{ , where each } p_i \text{ is a probability } d(x_i) \in [0, 1].\} \\
\mathcal{D}(f) &: \mathcal{D}(X) \to \mathcal{D}(Y) \\
&\mathcal{D}(f)(\textstyle\sum p_i |x_i\rangle) = \textstyle\sum p_i |f(x_i)\rangle
\end{aligned}$$

- $\eta_X : X \to \mathcal{D}(X)$, $\mu_X : \mathcal{D}(\mathcal{D}(X)) \to \mathcal{D}(X)$, such that

$$\eta_X(x) = 1 \cdot |x\rangle \,, \qquad \mu_X\left(\sum_{i \in I} a_i \left| \sum_{x \in X} b_{ix} |x\rangle \right\rangle\right) = \sum_{x \in X} \left(\sum_{i \in I} a_i b_{ix}\right) |x\rangle$$

  例えば $\left(\frac{1}{3}\left|\frac{1}{2}|x\rangle + \frac{1}{2}|y\rangle\right\rangle + \frac{2}{3}\,||z\rangle\rangle\right) \mapsto \left(\frac{1}{6}|x\rangle + \frac{1}{6}|y\rangle + \frac{2}{3}|z\rangle\right)$ となり、2 段階の遷移を 1 段階に潰します。



なお「**劣分布**」(sub-distribution) とは、遷移確率が 1 以下である確率分布のことです。たとえば、「x_0 から、確率 0.4 で x_1 に遷移し、確率 0.3 で x_2 に遷移し、それ以外は異常停止 (deadlock) する」といった、合計が 1 よりも小さい遷移確率も認めるものです。（これは理論的にも便利で、sub-distribution の集合は ω-cpo になります。）

さて、モナドの中でも「強モナド」と呼ばれるものがよく知られています。

定義 20（強モナド）　カルテシアン閉圏 $(\mathbb{C}, \otimes, I)$ 上で定義される**左-強モナド** (left-strong monad) とは、モナド (T, η, μ) すなわち任意の $X \in \mathbb{C}$ について 1) $T(X) \in \mathbb{C}$, 2) $\eta_X : X \to T(X)$, 3) $\mu_X : T(T(X)) \to T(X)$ の 3 つ組であって、さらにそれに加えて 4) **左-ストレングス** (left-strength)

$$str^L_{X,Y} : X \otimes T(Y) \to T(X \otimes Y)$$

を持ち、以下の図式を可換にするものである。

$$\begin{tikzcd}
I \otimes T(X) \arrow[rr, "str_{I,X}"] \arrow[dr, "\cong"'] & & T(I \otimes A) \arrow[dl, "\cong"] \\
& T(X) &
\end{tikzcd}
\qquad
\begin{tikzcd}
(X \times Y) \otimes T(Z) \arrow[rr, "str_{A\otimes B,C}"] \arrow[d, "\cong"'] & & T((X \otimes Y) \otimes Z) \arrow[d, "\cong"] \\
X \otimes (Y \otimes T(Z)) \arrow[dr, "\mathrm{id}_X \otimes str_{Y,Z}"'] & & T(X \otimes (Y \otimes Z)) \\
& X \otimes T(Y \otimes Z) \arrow[ur, "str_{X,Y\otimes Z}"'] &
\end{tikzcd}$$

$$\begin{tikzcd}
& X \otimes Y \arrow[dl, "\mathrm{id}_X \otimes \eta_Y"'] \arrow[dr, "\eta_{X\otimes Y}"] & \\
X \otimes T(Y) \arrow[rr, "str_{X,Y}"'] & & T(X \otimes Y)
\end{tikzcd}
\qquad
\begin{tikzcd}
& T(X \otimes T(Y)) \arrow[dr, "T(str_{X,Y})"] & \\
X \otimes T(T(Y)) \arrow[ur, "str_{X,T(Y)}"] \arrow[d, "\mathrm{id}_X \otimes \mu_Y"'] & & T(T(X \otimes Y)) \arrow[d, "\mu_{X\otimes Y}"] \\
X \otimes T(Y) \arrow[rr, "str_{X,Y}"'] & & T(X \otimes Y)
\end{tikzcd}$$

同様に、$(\mathbb{C}, \otimes, I)$ 上で定義される**右-強モナド** (right-strong monad) とは、**右-ストレングス** (right-strength)

$$str^R_{X,Y} : T(X) \otimes Y \to T(X \otimes Y)$$

を持つモナドであって、以下の図式を可換にするものである。

……

補足　**Set** 上のモナドは強モナドである。（**Set** を、直積 $\times$ をテンソル積として持った対称モノイダル圏と見る！）

強モナドのうち、特に可換モナドと呼ばれるものがあります。

定義 21（可換モナド）　**可換モナド**とは、**二重-ストレングス** (double-strength) $T(X) \otimes T(Y) \to T(X \otimes Y)$ を持った強モナドである。なお、二重-ストレングスとは、以下の図式のように、左右のストレングスから構成され、$\mu \circ T(str^L) \circ str^R = \mu \circ T(str^R) \circ str^L$ というふうに一致するものをいう。（可換性）

$$\begin{tikzcd}
 & T(X \otimes T(Y)) \arrow[r, "T(str^L)"] & T(T(X \otimes Y)) \arrow[dr, "\mu"] & \\
T(X) \otimes T(Y) \arrow[ur, "str^R"] \arrow[dr, "str^L"'] & & & T(X \otimes Y) \\
 & T(T(X) \otimes Y) \arrow[r, "T(str^R)"] & T(T(X \otimes Y)) \arrow[ur, "\mu"'] &
\end{tikzcd}$$

例 22（モノイド計算効果）　モノイド M に対し、$T := M \otimes (-)$ はモナドをなす。

- モナドの単位 $X \xrightarrow{\eta_X} M \otimes X$ は、
$$X \xrightarrow{\cong} I \otimes X \xrightarrow{e \otimes X} M \otimes X$$
で与えられる。この $I \xrightarrow{e} M$ はモノイドの単位。

- モナドの乗法 $(M \otimes M) \otimes X \xrightarrow{\mu_X} M \otimes X$ は、
$$(M \otimes M) \otimes X \xrightarrow{m \otimes X} M \otimes X$$
で与えられる。この $M \otimes M \xrightarrow{m} M$ はモノイドの乗法。

ここで、このモノイド M を**副作用** (side effect) と呼ぶことにする。

さて、この (T, η, μ) がモナドをなすことの恩恵は、大きく分けて 2 つある。

1. ただの関数 (pure function) $X \to Y$ を、効果つき計算 (effectful computation) $X \to TY$ とみなせる。
$$\begin{tikzcd}
X \arrow[r, "f"] & Y \arrow[d, "\eta_Y"] \\
 & M \otimes Y
\end{tikzcd}$$
2. 計算効果 $X \to TY$ と $Y \to TZ$ を集約して、$X \to TZ$ に合成することができる。
$$\begin{tikzcd}
X \arrow[r, "f"] & M \otimes Y \arrow[r, "Tg"] & M \otimes (M \otimes Z) \arrow[r, "\text{assoc}"] & (M \otimes M) \otimes Z \arrow[d, "\mu_Z"] \\
 & & & M \otimes Z
\end{tikzcd}$$

ちなみに、$T = M \otimes (-)$ が **可換モナド** であるのは、M が **可換モノイド** であるとき、かつそのときに限る。

$$\begin{tikzcd}
 & \overset{1}{\smile}M \otimes (X \otimes (\overset{2}{\smile}M \otimes Y)) \arrow[r] & \overset{1}{\smile}M \otimes (\overset{2}{\smile}M \otimes (X \otimes Y)) \arrow[dr] & \\
(\overset{1}{\smile}M \otimes X) \otimes (\overset{2}{\smile}M \otimes Y) \arrow[ur] \arrow[dr] & & & M \otimes (X \otimes Y) \\
 & \overset{2}{\smile}M \otimes ((\overset{1}{\smile}M \otimes X) \otimes Y) \arrow[r] & \overset{2}{\smile}M \otimes (\overset{1}{\smile}M \otimes (X \otimes Y)) \arrow[ur] &
\end{tikzcd}
\quad \overset{\text{iff}}{\Longleftrightarrow} \quad
\begin{tikzcd}
\overset{1}{\smile}M \otimes \overset{2}{\smile}M \arrow[rr, "\text{commutative}", "\cong"'] \arrow[dr, "m"'] & & \overset{2}{\smile}M \otimes \overset{1}{\smile}M \arrow[dl, "m"] \\
 & M &
\end{tikzcd}$$

補足　上の**例 22** が非可換モナドの場合、これをライターモナド (writer monad) $T = \Sigma^* \times (-)$ と呼ぶ。
直観的には、各実行ごとに文字を吐き出していき、それを順に並べていって文字列にして出力するモナドである。

ではここで、「**クライスリ圏**」と「**アイレンバーグ・ムーア圏**」を導入します。これらは、モナドと密接な関係を持ち、コンピューターサイエンスの中でたくさん使われます。

定義 23（クライスリ圏） $T : \mathbb{C} \to \mathbb{C}$ を、圏 $\mathbb{C}$ 上の自己関手とする。モナド (T, η, μ) が与えられたとき、以下で定められた圏 $\mathbb{C}_T$ を、**クライスリ圏** (Kleisli category) と呼ぶ。

- $\mathbb{C}_T$ における対象 $X, Y, \ldots$ は、$\mathbb{C}$ における対象 $X, Y, \ldots$ とする。
- $\mathbb{C}_T$ における射 $f : X \nrightarrow Y$ は、$\mathbb{C}$ における射 $f : X \to TY$ とする。

$$\frac{f : X \nrightarrow Y \text{ in } \mathbb{C}_T}{\overline{f : X \to TY \text{ in } \mathbb{C}}}$$

- 射の合成： $\mathbb{C}_T$ の射 $f : X \to Y$ と $g : Y \to Z$ に対し、合成 $g \circ f : X \to Z$ は、以下で定義される。

$$\frac{X \xrightarrow{f} Y \xrightarrow{g} Z \qquad (\text{in } \mathbb{C}_T)}{\dfrac{\begin{array}{ll} X \xrightarrow{f} T(Y) & (\text{in } \mathbb{C}) \\ Y \xrightarrow{g} T(Z) & \end{array}}{X \xrightarrow{f} T(Y) \xrightarrow{T(g)} T(T(Z)) \xrightarrow{\mu_Z} T(Z) \qquad (\text{in } \mathbb{C})}}$$

- 恒等射： $\mathbb{C}_T$ における射 $\mathrm{id}_X : X \to X$ は、$\mathbb{C}$ における射 $\eta_X : X \to T(X)$ で定義される。

注意 圏 $\mathbb{C}$ における任意の射 $f : X \to TY$ に対し、$(-)^* : \mathrm{Hom}_{\mathbb{C}}(X, TY) \to \mathrm{Hom}_{\mathbb{C}}(TX, TY)$ を、

$$f^* = \mu_Y \circ Tf$$

と定める。この表記を用いると、$\mathbb{C}_T$ における射 $f \in \mathrm{Hom}_{\mathbb{C}_T}(X, Y)$ と射 $g \in \mathrm{Hom}_{C_T}(Y, Z)$ の合成射 $g \circ f$ は、$\mathbb{C}$ における射 $g^* \circ f : X \to TZ$ のことである。

定義 24（EM 圏） $T : \mathbb{C} \to \mathbb{C}$ を、圏 $\mathbb{C}$ 上の自己関手とする。モナド (T, η, μ) が与えられたとき、以下で定められた圏 $\mathbb{C}^T$ を、**アイレンバーグ・ムーア圏** (Eilenberg-Moore category) と呼ぶ。

- $\mathbb{C}^T$ における対象は、T-代数 $(TX \xrightarrow{a} X)$ である。
- $\mathbb{C}^T$ における射は、T-代数どうしの間の準同型である。

$$\frac{\begin{pmatrix} TX \\ \downarrow a \\ X \end{pmatrix} \xrightarrow{f} \begin{pmatrix} TY \\ \downarrow b \\ Y \end{pmatrix} \text{ in } \mathbb{C}^T}{\overline{X \xrightarrow{f} Y \text{ in } \mathbb{C}}}$$

- そして η と μ に整合的である。

$$\begin{tikzcd} X \arrow[r, "\eta_X"] \arrow[dr, "\mathrm{id}_X"'] & TX \arrow[d, "a"] \\ & X \end{tikzcd} \qquad \begin{tikzcd} T^2X \arrow[r, "\mu_X"] \arrow[d, "Ta"'] & TX \arrow[d, "a"] \\ TX \arrow[r, "a"'] & X \end{tikzcd}$$

ここで、モナド、クライスリ圏、アイレンバーグ・ムーア圏を考えることのモチベーションをつかむために、例として、**例外処理** (exception handler) の例を見ておくことにしましょう。

- まず、**例外モナド** (exception monad) は以下のように与えられます。
  - $T : \mathbf{Set} \to \mathbf{Set}$ を、$T(X) := X + E$ と定めます。ここで $+$ は余積とし、E は "エラーの集合" とします。さらに、$f : X \to Y$ については、$T(f) = f + \mathrm{id}_E$ とします。
  - ここで、単位 η_X を以下のように定義します。(「もしエラーがなければ、計算結果をそのまま埋め込む。」)

$$\begin{array}{rccccc} \eta_X : & X & \longrightarrow & X & + & E \\ & x & \mapsto & x & & \end{array} \qquad \Big(\eta_X(x) \ := inl(x) \Big)$$

  - また、乗法 μ_X を以下のように定義します。(「一度でもエラーが起きたら、そのエラーを返す。」)

$$\begin{array}{rccccccccc} \mu_X : & (& X & + & E &) & + & E & \longrightarrow & X & + & E \\ & & x & & & & & & \mapsto & x & & \\ & & & & e & & & & \mapsto & & & e \\ & & & & & & e & & \mapsto & & & e \end{array} \qquad \left(\begin{array}{rl} \mu_X(inl(inl(x))) & := inl(x) \\ \mu_X(inl(inr(e))) & := inr(e) \\ \mu_X(inr(e)) & := inr(e) \end{array} \right)$$

  - このようにして、(T, η, μ) はモナドをなします。
- 次に、$f : X \to Y + E$ と $g : Y \to Z + E$ に対し、それらのクライスリ合成 (Kleisli composition) は

$$g * f := \mu_Z \circ Tg \circ f$$

で定まります。(「もしエラーが起きたら、そのエラーを返して、次の計算はスキップする。もしエラーが起きなければ、その計算結果を送ってから、そのまま次の計算も続行する。」)

$$\begin{array}{ccccccccccccc} X & \xrightarrow{f} & Y & + & E & \xrightarrow{Tg} & (& Z & + & E &) & + & E & \xrightarrow{\mu_Z} & Z & + & E \\ x & \mapsto & y & & & \mapsto & & z & & & & & & \mapsto & z & & \\ x & \mapsto & y & & & \mapsto & & & & e & & & & \mapsto & & & e \\ x & \mapsto & & & e & \mapsto & & & & & & & e & \mapsto & & & e \end{array}$$

- さらに、このモナド (T, η, μ) のもとで、以下の T-代数 $X + E \xrightarrow{h} X$ を「**例外処理**」(exception handler) と呼びます。(「すべての例外を最終的に処理し、回復して通常値に戻す。」)

$$\begin{tikzcd} X \arrow[r, "\eta_X"] \arrow[dr, "\mathrm{id}_X"'] & X + E \arrow[d, "h"] \\ & X \end{tikzcd} \qquad \begin{tikzcd} (X + E) + E \arrow[r, "\mu_X"] \arrow[d, "T(h)"'] & X + E \arrow[d, "h"] \\ X + E \arrow[r, "h"'] & X \end{tikzcd}$$

なお、この h を与えることは、「各例外 e に対して、回復値 $k(e) \in X$ を選ぶこと」と実質的に同じです。

$$k : E \to X, \quad k(e) := h(inr(e))$$

$$\text{i.e.,} \quad h = [\mathrm{id}_X, k] : X + E \to X$$

まさにこの例こそ、モナド、クライスリ圏、アイレンバーグ・ムーア圏を考えることの嬉しさがよくわかる、最もシンプルで典型的な例だと言えるでしょう！

ちょうどいい機会なので、「**前モノイダル直積**」(pre-monoidal product) という考え方にも触れておきましょう。

補足　いま、$\mathbb{C}$ をカルテシアン閉圏とし、$T : \mathbb{C} \to \mathbb{C}$ を、左ストレングス $str^L_{X,Y} : X \otimes T(Y) \to T(X \otimes Y)$ と右ストレングス $str^R_{X,Y} : T(X) \otimes Y \to T(X \otimes Y)$ を持つ強モナドとします。このとき、クライスリ圏 $\mathbb{C}_T$ は

- 対象： 圏 $\mathbb{C}$ における対象と同じ
- 射： 圏 $\mathbb{C}_T$ における射 $X \to X'$ は、圏 $\mathbb{C}$ における射 $X \to T(X')$ として与えられる

といったものでした（ここまでは復習）。

ここで、クライスリ圏 $\mathbb{C}_T$ における直積が、一般には「モノイダルな直積」ではなく、「前モノイダルな直積」をなすことに注意が必要です。具体的に言うと、圏 $\mathbb{C}_T$ における直積 $f \otimes g$（射の並列合成）の構成

$$\begin{tikzcd} X \arrow[r, "f"] & X' \\ \otimes & \otimes \\ Y \arrow[r, "g"] & Y' \end{tikzcd} \qquad \boxed{\text{in } \mathbb{C}_T}$$

は、どちらの射を先に適用するかに依存してしまいます。というのも、

$$\begin{aligned} X \otimes Y &\xrightarrow{f \otimes \mathrm{id}} T(X') \otimes Y \xrightarrow{str^R} T(X' \otimes Y) \qquad \boxed{\text{in } \mathbb{C}} \\ &\xrightarrow{T(\mathrm{id} \otimes g)} T(X' \otimes T(Y')) \xrightarrow{T(str^L)} TT(X' \otimes Y') \\ &\xrightarrow{\mu} T(X' \otimes Y') \end{aligned}$$

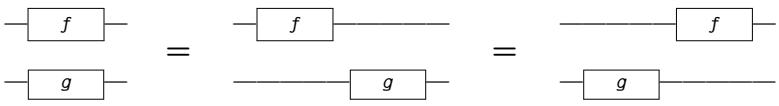
と

$$\begin{aligned} X \otimes Y &\xrightarrow{\mathrm{id} \otimes g} X \otimes T(Y') \xrightarrow{str^L} T(X \otimes Y') \qquad \boxed{\text{in } \mathbb{C}} \\ &\xrightarrow{T(f \otimes \mathrm{id})} T(T(X') \otimes Y') \xrightarrow{T(str^R)} TT(X' \otimes Y') \\ &\xrightarrow{\mu} T(X' \otimes Y') \end{aligned}$$

の両者は一般に異なります。このように、クライスリ圏における直積は、射を適用する順番に敏感なのです。

そこでクイズです。クライスリ圏の直積がモノイダルであるのはどんなときでしょうか？（考えてみてください。）答えは、T が可換な強モナドであるときです！ つまり、二重ストレングス $T(X) \otimes T(Y) \to T(X \otimes Y)$ があるとき、クライスリ圏 $\mathbb{C}_T$ はモノイダルになります。（すなわち、ストリング図式で書ける！）

$$f \otimes g \quad = \quad (\mathrm{id} \otimes g) \circ (f \otimes \mathrm{id}) \quad = \quad (f \otimes \mathrm{id}) \circ (\mathrm{id} \otimes g) \qquad \boxed{\text{in } \mathbb{C}_T}$$



これはなぜかというと、モナド T が可換であるとき、

$$\begin{tikzcd} X \arrow[r, "f"] & T(X') \arrow[r, equal] & T(X') \arrow[dr] & \\ \otimes & \otimes & \otimes & T(X' \otimes Y') \\ Y \arrow[r, equal] & Y \arrow[r, "g"] & T(Y') \arrow[ur] & \end{tikzcd} \qquad \boxed{\text{in } \mathbb{C}}$$

と

$$\begin{tikzcd} X \arrow[r, equal] & X \arrow[r, "f"] & T(X') \arrow[dr] & \\ \otimes & \otimes & \otimes & T(X' \otimes Y') \\ Y \arrow[r, "g"] & T(Y') \arrow[r, equal] & T(Y') \arrow[ur] & \end{tikzcd} \qquad \boxed{\text{in } \mathbb{C}}$$

の両者は一致するからです！

さて、ここからは、しばらく「**モナド分配則**」について取り上げます。(補足: 実用上は、F のほうはモナドでなくてただの関手で済む状況もあり、その場合は、下の定義のうち左から２番目と４番目の図式の条件は要らなくなります。)

定義 25（モナド分配則） モナド分配則 (distributive law) とは、２つのモナド (T, η^T, μ^T) と (F, η^F, μ^F) の間の自然変換 $\lambda : FT \Rightarrow TF$ であって、以下の図式を可換にするものである。

$$\begin{tikzcd} FT^2 \arrow[r, "\lambda T"] \arrow[d, "F\mu^T"'] & TFT \arrow[r, "T\lambda"] & T^2F \arrow[d, "\mu^T F"] \\ FT \arrow[rr, "\lambda"'] & & TF \end{tikzcd} \qquad \begin{tikzcd} F^2T \arrow[r, "F\lambda"] \arrow[d, "\mu^F T"'] & FTF \arrow[r, "\lambda F"] & TF^2 \arrow[d, "T\mu^F"] \\ FT \arrow[rr, "\lambda"'] & & TF \end{tikzcd} \qquad \begin{tikzcd} & F \arrow[dl, "F\eta^T"'] \arrow[dr, "\eta^T F"] & \\ FT \arrow[rr, "\lambda"'] & & TF \end{tikzcd} \qquad \begin{tikzcd} & T \arrow[dl, "\eta^F T"'] \arrow[dr, "T\eta^F"] & \\ FT \arrow[rr, "\lambda"'] & & TF \end{tikzcd}$$

まずは、この「モナド分配則」を具体的にどうやって使うかを、以下ではオートマトンの例を挙げて確認します。

(1) Kleisli approach

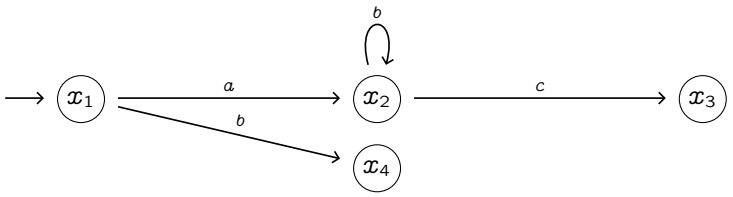
- 状態遷移システム $\gamma : X \to TFX$ を考えます。
  - T: 分岐をあらわす冪集合モナド $\mathcal{P}(-)$
  - F: １ステップ遷移をあらわす関手 $1 + \Sigma \times (-)$
  - つまり、このシステムは $\gamma : X \to \mathcal{P}(1 + \Sigma \times X)$ という余代数で、$\gamma(x) = \{(a, x'), \ldots\}$ といったもの。
- ここで、モナド分配則 $\lambda : FT \Rightarrow TF$ を考えます。
- すると、以下のように**合成**を考えることができます。(ここで $X \to TF^2X$ が「**2 ステップ遷移**」を表します。)

$$\begin{tikzcd} X \arrow[r, "\gamma"] & TFX \\ & X \arrow[r, "\gamma"] & TFX \end{tikzcd}$$

$$\begin{tikzcd} X \arrow[r, "\gamma"] & TFX \arrow[r, "TF\gamma"] & TFTFX \arrow[d, "T\lambda_{FX}"] \\ & & TTFFX \arrow[d, "\mu_{F^2X}"] \\ & & TF^2X \end{tikzcd}$$

$$X \longrightarrow TF^2X$$

- たとえば、以下のようになります。



$$\begin{tikzcd} x_1 \arrow[r, mapsto] & \gamma(x_1) \arrow[d, equal] \\ & \{(a, x_2), (b, x_4)\} \arrow[r, mapsto] & \{(a, \gamma(x_2)), (b, \gamma(x_4))\} \arrow[d, equal] \\ & & \{(a, \{(b, x_2), (c, x_3)\}), (b, \{\})\} \arrow[d, mapsto, "T\lambda_{FX}"] \\ & & \{\{(ab, x_2), (ac, x_3)\}, \{\}\} \arrow[d, mapsto, "\mu_{F^2X}"] \\ & & \{(ab, x_2), (ac, x_3)\} \end{tikzcd}$$

(2) Eilenberg-Moore approach

- 状態遷移システム $\gamma : X \to FTX$ を考えます。
  - T: 分岐をあらわす冪集合モナド $\mathcal{P}(-)$
  - F: 1 ステップ遷移をあらわす関手 $2 \times (-)^{\Sigma}$
  - つまり、このシステムは $\gamma : X \to 2 \times \mathcal{P}(X)^{\Sigma}$ という余代数で、$\gamma(x)(a) = \{x', \ldots\}$ といったもの。
- ここで、モナド分配則 $\lambda' : TF \Rightarrow FT$ を考えます。
- すると、以下のように、非決定性オートマトンを**決定化** (determinize) することができます。

$$\begin{tikzcd} TX \arrow[r, "T\gamma"] & TFTX \arrow[r, "\lambda'_{TX}"] & FTTX \arrow[d, "F\mu_X"] \\ & & F(TX) \end{tikzcd}$$

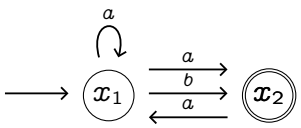
この $TX \to F(TX)$ は、まさに「**冪集合構成**」(powerset construction) に相当します。
詳しく言うと、以下の合成を通じて、

$$\begin{tikzcd} \mathcal{P}(X) \arrow[r, "\mathcal{P}\gamma"] & \mathcal{P}(2 \times \mathcal{P}(X)^{\Sigma}) \arrow[r, "\lambda'_{\mathcal{P}(X)}"] & 2 \times \mathcal{P}(\mathcal{P}(X))^{\Sigma} \arrow[d, "2\times(\mu_X)^{\Sigma}"] \\ & & 2 \times \mathcal{P}(X)^{\Sigma} \end{tikzcd}$$

$\delta(S)(a) = \bigcup_{x \in S} \gamma(x)(a)$ という決定化 (determinization) を得ることができます。

$$\begin{array}{rccc} \gamma : & X & \longrightarrow & 2 \times \mathcal{P}(X)^{\Sigma} \\ \text{“determinize”} \rightsquigarrow & & & \\ \delta : & \mathcal{P}(X) & \longrightarrow & 2 \times \mathcal{P}(X)^{\Sigma} \end{array}$$

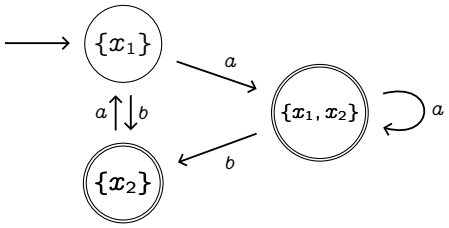
- たとえば、以下の非決定性オートマトン $\gamma : X \to 2 \times \mathcal{P}(X)^{\Sigma}$ は、



$$\gamma(x_1) = \begin{bmatrix} a & \mapsto & \{x_1, x_2\} \\ b & \mapsto & \{x_2\} \end{bmatrix}$$

$$\gamma(x_2) = \begin{bmatrix} a & \mapsto & \{x_1\} \\ b & \mapsto & \emptyset \end{bmatrix}$$

$\delta = (F\mu_X \circ \lambda'_{TX} \circ T\gamma) : \mathcal{P}(X) \to 2 \times \mathcal{P}(X)^{\Sigma}$ により、決定性オートマトンになります。



$$\delta(\{x_1\}) = \begin{bmatrix} a & \mapsto & \{x_1, x_2\} \\ b & \mapsto & \{x_2\} \end{bmatrix}$$

$$\delta(\{x_2\}) = \begin{bmatrix} a & \mapsto & \{x_1\} \\ b & \mapsto & \emptyset \end{bmatrix}$$

$$\delta(\{x_1, x_2\}) = \begin{bmatrix} a & \mapsto & \{x_1, x_2\} \\ b & \mapsto & \{x_2\} \end{bmatrix}$$

※ なお、両者 (1) $\gamma : X \to \mathcal{P}(1 + \Sigma \times X)$ と (2) $\gamma : X \to 2 \times \mathcal{P}(X)^{\Sigma}$ は同じものです（自然同型で対応)。

$$\begin{array}{c} X \to \mathcal{P}(1 + \Sigma \times X) \\ \hline\hline \text{relation } R \subseteq X \times (1 + \Sigma \times X) \\ \hline\hline \begin{array}{cc} \text{relation } R_1 \subseteq X \times 1 & \text{relation } R_2 \subseteq X \times \Sigma \times X \\ \hline\hline \text{relation } R_1 \subseteq X & X \times \Sigma \to \mathcal{P}(X) \\ \hline\hline X \to 2 & X \to \mathcal{P}(X)^{\Sigma} \end{array} \\ \hline\hline X \to 2 \times \mathcal{P}(X)^{\Sigma} \end{array}$$

上記で挙げた２種類の異なる見方 **(1) Kleisli approach** と **(2) Eilenberg-Moore approach** は、それぞれ

(1) 分配則 $\lambda : FT \Rightarrow TF$ によって誘導される F の **Kleisli extension** F_T と
(2) 分配則 $\lambda' : TF \Rightarrow FT$ によって誘導される F の **Eilenberg-Moore lifting** F^T

に基づくものだと言えます。このことについて、以下で詳しく見ていきます。

まずは、準備として、以下の自由忘却随伴 (free-forgetful adjunction) について確認しておきましょう。

命題 26　クライスリ圏と EM 圏について、それぞれ、以下のような自由忘却随伴がある。（証明略）

$$\begin{tikzcd} \mathbb{C}_T \arrow[d, bend left, "R_T"] \arrow[d, phantom, "\dashv"] \\ \mathbb{C} \arrow[u, bend left, "L_T"] \end{tikzcd} \qquad\qquad \begin{tikzcd} \mathbb{C}^T \arrow[d, bend left, "R^T"] \arrow[d, phantom, "\dashv"] \\ \mathbb{C} \arrow[u, bend left, "L^T"] \end{tikzcd}$$

$$L_T \dashv R_T \qquad\qquad L^T \dashv R^T$$

ここで、**クライスリ拡大** F_T と **アイレンバーグ・ムーア持ち上げ** F^T の、両者を定義します。

定義 27（Kleisli extension）　F_T が F の**クライスリ拡大** (Kleisli extension) であるとは、

$$\begin{tikzcd} \mathbb{C}_T \arrow[r, "F_T"] & \mathbb{C}_T \\ \mathbb{C} \arrow[u, "L_T"] \arrow[r, "F"'] & \mathbb{C} \arrow[u, "L_T"'] \end{tikzcd} \qquad\qquad \begin{aligned} F_T L_T &= L_T F \\ \eta^{F_T} L_T &= L_T \eta^F \\ \mu^{F_T} L_T &= L_T \mu^F \end{aligned}$$

を満たすことである。

定義 28（EM lifting）　F^T が F の**アイレンバーグ・ムーア持ち上げ** (Eilenberg-Moore lifting) であるとは、

$$\begin{tikzcd} \mathbb{C}^T \arrow[r, "F^T"] \arrow[d, "R^T"] & \mathbb{C}^T \arrow[d, "R^T"] \\ \mathbb{C} \arrow[r, "F"'] & \mathbb{C} \end{tikzcd} \qquad\qquad \begin{aligned} R^T F^T &= F R^T \\ R^T \eta^{F^T} &= \eta^F R^T \\ R^T \mu^{F^T} &= \mu^F R^T \end{aligned}$$

を満たすことである。

（補足:　なお、F のほうをモナドではなく単なる関手と考える場合には、上の**定義 27** と**定義 28** の、２つ目と３つ目の条件をなくし、単に「上記の図式が可換になること」だけをもって定義とします。）

ここで、分配則 λ, λ' と、拡大 F_T と持ち上げ F^T が、互いに bijective に誘導し合うことを確認しましょう。

補足　モナド分配則 $\lambda : FT \Rightarrow TF$ は、
F のクライスリ拡大 F_T を誘導する。

$$\begin{tikzcd} \mathbb{C}_T \arrow[r, "F_T"] & \mathbb{C}_T \\ X \arrow[d, "f"'] & FX \arrow[d, "Ff"] \\ Y & FTY \arrow[d, "\lambda_Y"] \\ & TFY \end{tikzcd} \qquad \longmapsto$$

モナド分配則 $\lambda' : TF \Rightarrow FT$ は、
F の EM 持ち上げ F^T を誘導する。

$$\begin{tikzcd} \mathbb{C}^T \arrow[r, "F^T"] & \mathbb{C}^T \\ \Psi & \Psi \\ \begin{pmatrix} TX \\ \downarrow a \\ X \end{pmatrix} \arrow[r, mapsto] & \begin{pmatrix} TFX \\ \downarrow \lambda'_X \\ FTX \\ \downarrow Fa \\ FX \end{pmatrix} \end{tikzcd}$$

補足　F のクライスリ拡大 F_T は、
モナド分配則 $\lambda : FT \Rightarrow TF$ を誘導する。

$$\begin{tikzcd} TX \arrow[d, "f"'] & FTX \arrow[d, "F_T(\mathrm{id}_{TX})"] \\ X \arrow[r, mapsto, "F_T"] & TFX \end{tikzcd}$$

F の EM 持ち上げ F^T は、
モナド分配則 $\lambda' : TF \Rightarrow FT$ を誘導する。

$$L^T X = \begin{pmatrix} TTX \\ \downarrow \mu_X \\ TX \end{pmatrix} \xmapsto{F^T} \begin{pmatrix} TFX \\ \downarrow TF\eta_X \\ TFTX \\ \downarrow \varphi_X \\ FTX \end{pmatrix}$$

ここで、先ほどの (1) Kleisli approach と (2) Eilenberg-Moore approach は、数学的主張として正確に述べると、それぞれ次のようなもの（「自由関手の持ち上げ」）として理解されます。

命題 29（Kleisli approach）　分配則 $\lambda : FT \Rightarrow TF$ が与えられたとき、L_T は $\widehat{L_T}$ に持ち上がる。

$$\begin{tikzcd} \mathrm{Coalg}_{\mathbb{C}}(TF) \arrow[r, shift left, "\widehat{L_T}"] \arrow[d] \arrow[r, phantom, "\perp"] & \mathrm{Coalg}_{\mathbb{C}_T}(F_T) \arrow[l, shift left, "\widehat{R_T}"] \arrow[d] \\ \mathbb{C} \arrow[loop left, "F"] \arrow[loop below, "T"] \arrow[r, shift left, "L_T"] \arrow[r, phantom, "\perp"] & \mathbb{C}_T \arrow[l, shift left, "R_T"] \arrow[loop right, "F_T"] \end{tikzcd}$$

命題 30（EM approach）　分配則 $\lambda' : TF \Rightarrow FT$ が与えられたとき、L^T は $\widehat{L^T}$ に持ち上がる。

$$\begin{tikzcd} \mathrm{Coalg}_{\mathbb{C}}(FT) \arrow[r, shift left, "\widehat{L^T}"] \arrow[d] \arrow[r, phantom, "\perp"] & \mathrm{Coalg}_{\mathbb{C}^T}(F^T) \arrow[l, shift left, "\widehat{R^T}"] \arrow[d] \\ \mathbb{C} \arrow[loop left, "F"] \arrow[loop below, "T"] \arrow[r, shift left, "L^T"] \arrow[r, phantom, "\perp"] & \mathbb{C}^T \arrow[l, shift left, "R^T"] \arrow[loop right, "F^T"] \end{tikzcd}$$

これだけだとやや抽象的でわかりにくいので、もう少し詳しく見ていきましょう。

まず、前者 $\boxed{\textbf{(1) Kleisli approach}}$ については、以下のように理解することができます。

関手 $\widehat{L_T} : \mathrm{Coalg}_{\mathbb{C}}(TF) \to \mathrm{Coalg}_{\mathbb{C}_T}(F_T)$ は、以下のように定めます。

- 対象に対しては、

$$\mathrm{Coalg}_{\mathbb{C}}(TF) \ni \begin{pmatrix} TFX \\ \uparrow \gamma \\ X \end{pmatrix} \xmapsto{\widehat{L_T}} \begin{pmatrix} FX \\ \uparrow\!\!\!

次に、後者 $\boxed{\textbf{(2) Eilenberg-Moore approach}}$ については、以下のように理解することができます。

関手 $\widehat{L^T} : \mathrm{Coalg}_{\mathbb{C}}(FT) \to \mathrm{Coalg}_{\mathbb{C}^T}(F^T)$ は、以下のように定めます。

- 対象に対しては、

$$\mathrm{Coalg}_{\mathbb{C}}(FT) \ni \begin{pmatrix} FTX \\ \uparrow \gamma \\ X \end{pmatrix} \xmapsto{\widehat{L^T}} \begin{pmatrix} F^TL^TX \\ \uparrow \overline{\gamma} \\ L^TX \end{pmatrix} \in \mathrm{Coalg}_{\mathbb{C}^T}(F^T)$$

とします。これは、自由忘却随伴 $L^T \dashv R^T$ より、

$$\begin{array}{ccc} \mathrm{Hom}_{\mathbb{C}}(X, R^TF^TL^TX) & \cong & \mathrm{Hom}_{\mathbb{C}^T}(L^TX, F^TL^TX) \\ \Psi & & \Psi \\ \gamma : X \to FTX & & \overline{\gamma} : L^TX \to F^TL^TX \end{array}$$

という対応関係から一意に得られるものであり、具体的には以下のように定まります。

$$\begin{array}{ccccccc} \overline{\gamma} := & L^TX & \xrightarrow{L^T\gamma} & L^TR^TF^TL^TX & \xrightarrow{\varepsilon_{(F^TL^TX)}} & F^TL^TX \\ & \| & & \| & & \| \\ & \begin{pmatrix} TTX \\ \downarrow \mu_X \\ TX \end{pmatrix} & \xrightarrow{T\gamma} & \begin{pmatrix} TTFTX \\ \downarrow \mu_{(FTX)} \\ TFTX \end{pmatrix} & \xrightarrow{\varphi_X} & \begin{pmatrix} TFTX \\ \downarrow \varphi_X \\ FTX \end{pmatrix} \end{array}$$

この $\overline{\gamma}$ は F^T-余代数であり、その underlying map は $R^T(\overline{\gamma}) : TX \to FTX$ となります。
なお、$\gamma = R^T(\overline{\gamma}) \circ \eta_X$ が、随伴による tranpose の性質から従います。(詳細略)

$$\begin{tikzcd} X \arrow[rr, "\eta_X"] \arrow[dr, "\gamma"'] & & TX \arrow[dl, "R^T(\overline{\gamma})"] \\ & FTX & \end{tikzcd}$$

ここで得られた $R^T(\overline{\gamma})$ は、以下の $\gamma^\sharp$ と一致します。(それゆえ、28 ページの冪集合構成に相当します。)

$$\gamma^\sharp = \left(TX \xrightarrow{T\gamma} TFTX \xrightarrow{\lambda'_{TX}} FTTX \xrightarrow{F\mu_X} FTX \right)$$

というのも、以下の図式は、外側がちゃんと可換になります。(右下は φ の自然性より。)

$$\begin{tikzcd} & TX \arrow[rr, "T\gamma"] \arrow[dl, "T\gamma"'] & & TFTX \arrow[dr, "\varphi_X"] \arrow[dl, "TF\mu_X"'] & \\ TFTX \arrow[rr, "TF\eta_{TX}"'] \arrow[rrr, bend left, "\mathrm{id}"] \arrow[dr, "\lambda'_{TX}"'] & & TFTTX \arrow[d, "\varphi_{TX}"] & & FTX \\ & & FTTX \arrow[urr, "F\mu_X"'] & & \end{tikzcd}$$

- 射に対しては、

$$\begin{pmatrix} FTX \\ \uparrow \gamma \\ X \end{pmatrix} \xrightarrow{f} \begin{pmatrix} FTY \\ \uparrow \delta \\ Y \end{pmatrix} \text{ in } \mathrm{Coalg}_{\mathbb{C}}(FT) \Longrightarrow \begin{pmatrix} F^TL^TX \\ \uparrow \overline{\gamma} \\ L^TX \end{pmatrix} \xrightarrow{L^Tf} \begin{pmatrix} F^TL^TY \\ \uparrow \overline{\delta} \\ L^TY \end{pmatrix} \text{ in } \mathrm{Coalg}_{\mathbb{C}^T}(F^T)$$

を以下の図式のように定めます。

$$\begin{tikzcd} FTX \arrow[r, "FTf"] & FTY \\ X \arrow[u, "\gamma"] \arrow[r, "f"'] & Y \arrow[u, "\delta"'] \end{tikzcd} \Longrightarrow \begin{tikzcd} F^TL^TX \arrow[r, "F^TL^Tf"] & F^TL^TY \\ L^TX \arrow[u, "\overline{\gamma}"] \arrow[r, "L^Tf"'] & L^TY \arrow[u, "\overline{\delta}"'] \end{tikzcd} \quad \text{given as} \quad \begin{tikzcd} FTX \arrow[rr, "FTf"] & & FTY \\ & TFTX \arrow[ul, "\varphi_X"'] \arrow[r, "TFTf"] & TFTY \arrow[ur, "\varphi_Y"] \\ TX \arrow[uu, "R^T\overline{\gamma}"] \arrow[ur, "T\gamma"] \arrow[rr, "Tf"'] & & TY \arrow[uu, "R^T\overline{\delta}"'] \arrow[ul, "T\delta"'] \end{tikzcd}$$

これら $\boxed{\textbf{(1) Kleisli approach}}$ と $\boxed{\textbf{(2) Eilenberg-Moore approach}}$ について、関連する話題を紹介します。

- (1) に関して、**軌跡意味論** (trace semantics) と呼ばれる考え方がある。

  ··· 圏 $\mathbb{C}$ における TF-余代数 $c : X \to TFX$ は、圏 $\mathbb{C}_T$ における F-余代数 $c : X \twoheadrightarrow FX$ である。
  ここで、圏 $\mathbb{C}$ での始 F-代数 $\alpha : FA \xrightarrow{\cong} A$ から、圏 $\mathbb{C}_T$ での終 F-余代数 $\eta_{FA} \circ \alpha^{-1} : A \twoheadrightarrow FA$ を作る！
  このとき、この終余代数への一意な射 $\mathrm{tr}_c : X \twoheadrightarrow A$（つまり $\mathrm{tr}_c : X \to TA$）を、軌跡意味論と呼ぶ。

$$\begin{tikzcd} FX \arrow[r, dashed, "F\mathrm{tr}_c"] & FA \\ X \arrow[u, "c"] \arrow[r, dashed, "\mathrm{tr}_c"'] & A \arrow[u, "\eta_{FA} \circ \alpha^{-1}"'] \end{tikzcd} \qquad \boxed{\text{in } \mathbb{C}_T}$$

  たとえば、余代数 $c : X \to \mathcal{P}(1 + \Sigma \times X)$ と、始代数 $\alpha : 1 + \Sigma \times \Sigma^* \xrightarrow{\cong} \Sigma^*$ に対し、
  軌跡意味論 $\mathrm{tr}_c : X \twoheadrightarrow \Sigma^*$（つまり $\mathrm{tr}_c : X \to \mathcal{P}(\Sigma^*)$）が得られる！

$$\begin{tikzcd} 1 + \Sigma \times X \arrow[r, dashed, "F\mathrm{tr}_c"] & 1 + \Sigma \times \Sigma^* \\ X \arrow[u, "c"] \arrow[r, dashed, "\mathrm{tr}_c"'] & \Sigma^* \arrow[u, "\eta_{1+\Sigma\times\Sigma^*} \circ \alpha^{-1}"'] \end{tikzcd} \qquad \boxed{\text{in } \mathbb{C}_{\mathcal{P}}}$$

- (2) に関して、このような**冪集合構成** (powerset construction) の恩恵は、以下のことができる点にもある。

  ··· 終 FT-余代数が作れない場合でも、終 T-余代数なら作れることがある。

$$\frac{c : X \to FTX,\ \text{もともとの } FT\text{-余代数}}{c' : TX \to TFX,\ \text{決定化された } T\text{-余代数}}$$

$$\begin{tikzcd} & FTX \arrow[r, dashed] & FZ \\ X \arrow[r, "\eta_X"'] & TX \arrow[u, "c'"] \arrow[r, dashed] & Z \arrow[u, "\zeta"', "\cong"] \end{tikzcd}$$

  たとえば、終 $2 \times (\mathcal{P}-)^{\Sigma}$-余代数は存在しないが、終 $2 \times (-)^{\Sigma}$-余代数は存在する！

$$\frac{c : X \to 2 \times (\mathcal{P}X)^{\Sigma},\ \text{非決定性オートマトン}}{c' : \mathcal{P}X \to 2 \times (\mathcal{P}X)^{\Sigma},\ \text{決定性オートマトン}}$$

$$\begin{tikzcd} & 2 \times (\mathcal{P}X)^{\Sigma} \arrow[r, dashed] & 2 \times (2^{\Sigma^*})^{\Sigma} \\ X \arrow[r, "\eta_X"'] & \mathcal{P}X \arrow[u, "c'"] \arrow[r, dashed] & 2^{\Sigma^*} \arrow[u, "\zeta"', "\cong"] \end{tikzcd}$$

ただし、$\mathcal{P}(\Sigma^*)$ のほうは covariant で、2^{Σ^*} のほうは contravariant であることに注意。

さて、以上のように、モナド分配則はあると便利なのですが、実際には、いつも都合良くモナド分配則が手に入るとは限りません。以下ではそのことについて、簡単な例をもとに考えてみましょう。（以下に挙げるような例は、自然変換にならないため、したがってモナド分配則の候補にもなれないことを確認しましょう。）

例 31　$\mathcal{P} : \mathbf{Set} \to \mathbf{Set}$ を冪集合モナドとする。このとき、以下の $\lambda : \mathcal{PP} \Rightarrow \mathcal{PP}$ は自然変換にならない。

$$\sum_{i\in I} a_i \left| \sum_{x\in X} b_{ix} \left|x\right\rangle \right\rangle \xmapsto{\lambda} \sum_{f\in X^I} \left(\prod_{i\in I} b_{i,f(i)} \right) \left| \sum_{i\in I} a_i \left|f(i)\right\rangle \right\rangle$$

たとえば、$X = \{a, b, c, d\}$, $Y = \{\alpha, \beta\}$ とし、

$$\begin{array}{rccc} f: & X & \longrightarrow & Y \\ & a & \mapsto & \alpha \\ & b & \mapsto & \beta \\ & c & \mapsto & \alpha \\ & d & \mapsto & \beta \end{array} .$$

とする。このとき、以下の図式は可換にならない。（自然性が成り立たない）

$$\begin{tikzcd}
\{\{a,b\},\{c,d\}\} \arrow[r, mapsto, "\lambda"] \arrow[d, mapsto, "PPf"'] & \{\{a,c\},\{a,d\}, \\ & \{b,c\},\{b,d\}\} \arrow[dr, mapsto, bend left, "PPf"] \\
\{\{\alpha,\beta\}\} \arrow[r, mapsto, "\lambda"'] & \{\{\alpha\},\{\beta\}\} \quad \neq \quad \{\{\alpha\},\{\alpha,\beta\},\{\beta\}\}
\end{tikzcd}$$

（なお、ケット記法では、$\{\{a,b\},\{c,d\}\}$ は、$\Big|\,|a\rangle + |b\rangle\Big\rangle + \Big|\,|c\rangle + |d\rangle\Big\rangle$ と書かれることに注意。）

他にも有名な例だと、以下のようなモナド分配則もないことが知られています。

例 32（Plotkin）　冪集合モナド $\mathcal{P} : \mathbf{Set} \to \mathbf{Set}$ と、分布モナド $\mathcal{D} : \mathbf{Set} \to \mathbf{Set}$ について、

$$\mathcal{DP} \Rightarrow \mathcal{PD}$$

や

$$\mathcal{PD} \Rightarrow \mathcal{DP}$$

は自然変換にならない。（詳細は省略）

Rewrite

3.7　双模倣

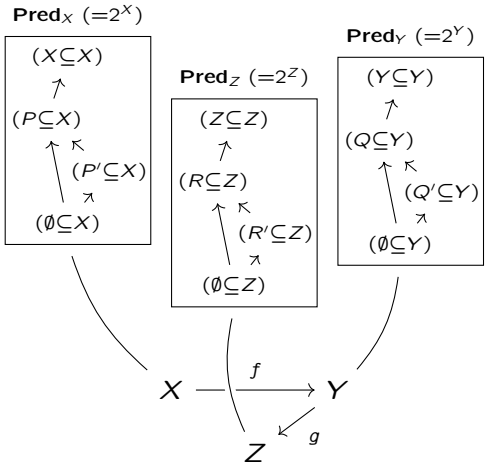
寄り道（余談）はこのくらいにしておいて、次節では、いよいよ（この記事の本題である）「グロタンディーク･ファイブレーション」の話に移りましょう。

3.8 グロタンディーク・ファイブレーション入門

グロタンディーク・ファイブレーションとは、手短に言うと、**論理学のモデルを、関数に沿って貼り合わせたもの**です。本記事のテーマであるモデル検査の文脈で言えば、各モデルを添字づけ、その添字に応じて関数に沿って引き戻せるようにすることで、**状態空間の違いをまたいで、仕様（検査したい性質）を一貫して追跡**できるようになります。

まずは直観を説明してから、後ほど正確な定義を述べます。例として $p : \mathbf{Pred} \to \mathbf{Set}$ というファイブレーションを考えてみましょう。ひとつずつステップを踏んで理解していきます。

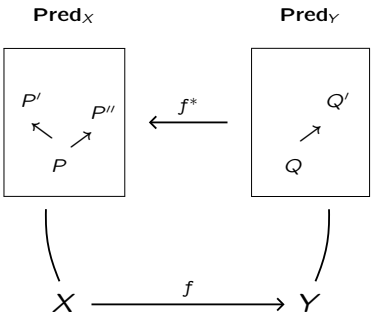
1. それぞれの対象 $X \in \mathbf{Set}$ に対して、「ファイバー」(fiber) と呼ばれる完備束 $\mathbf{Pred}_X$ $(:= 2^X)$ を割り当てます。



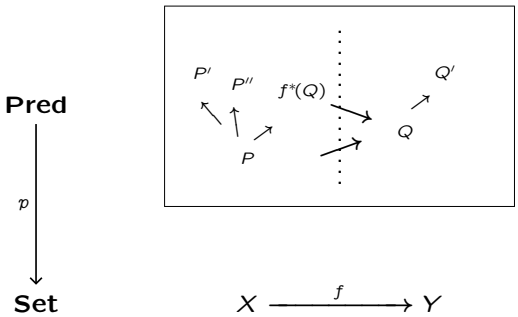
2. それぞれの射 $f : X \to Y$ に対して、$f^* : \mathbf{Pred}_Y \to \mathbf{Pred}_X$ を定めます。

$$
\begin{array}{cccc}
f^* : & 2^Y & \longrightarrow & 2^X \\
 & \rotatebox{90}{$\in$} & & \rotatebox{90}{$\in$} \\
 & (Y \xrightarrow{g} 2) & \longmapsto & (X \xrightarrow{f} Y \xrightarrow{g} 2)
\end{array}
$$

直観的には、以下の図のようなものだと理解するのがよいでしょう。



3. 以上で与えられたデータ $\left((\mathbf{Pred}_X)_{X \in \mathbf{Set}}, (f^* : \mathbf{Pred}_Y \to \mathbf{Pred}_X)_{f : X \to Y \text{ in } \mathbf{Set}}\right)$ をもとにして、各ファイバーを貼り合わせて、「全体圏」(total category) と呼ばれる圏 $\mathbf{Pred}$ をつくります。



4. この **Pred** は、対象が組 $(X,\ P \subseteq X)$ であり、射が写像 $(X,\ P \subseteq X) \to (Y,\ Q \subseteq Y)$ であるような圏です。
   - 対象:
   $$|\mathbf{Pred}| = \bigsqcup_{X \in \mathbf{Set}} |\mathbf{Pred}_X|$$
   - 射:
   $$\frac{(X,\ P \subseteq X) \longrightarrow (Y,\ Q \subseteq Y) \text{ in } \mathbf{Pred}}{\overline{\overline{X \xrightarrow{f} Y \text{ in } \mathbf{Set} \text{ s.t. } P \sqsubseteq f^*(Q) \text{ in } \mathbf{Pred}_X}}}$$
5. 以下の条件を満たす関手 $p : \mathbf{Pred} \to \mathbf{Set}$ は「ファイブレーション」(fibration) と呼ばれます。

$$\begin{tikzcd}
\mathbf{Pred} \arrow[d, "p"'] & Q \arrow[d, mapsto] \\
\mathbf{Set} & X \arrow[r, "f"'] & Y
\end{tikzcd}
\quad \Longrightarrow \quad
\begin{tikzcd}
{}^{\exists} f^*(Q) \arrow[r, "\exists \overline{f}(Q)"'] \arrow[d, mapsto] & Q \arrow[d, mapsto] \\
X \arrow[r, "f"'] & Y
\end{tikzcd}
\quad \text{s.t.} \quad
\begin{tikzcd}
R \arrow[d, mapsto] \arrow[dr, dashed, "\exists! s"] \arrow[drr, bend left, "\forall r"] & & \\
p(R) \arrow[r, "\forall g"'] \arrow[rr, bend right, "g \circ f"'] & f^*(Q) \arrow[r, "\overline{f}(Q)"'] \arrow[d, mapsto] & Q \arrow[d, mapsto] \\
& X \arrow[r, "f"'] & Y
\end{tikzcd}$$

6. ここで、**Pred** を「全体圏」といい、**Set** を「基底圏」といいます。
   また、$\overline{f}(Q) : f^*(Q) \to Q$ を、f の Q による「カルテシアン持ち上げ」(cartesian lifting) といいます。
   そして、f^* を、f の「置換」(substitution) といい、対象 $P, Q \in \mathbf{Pred}$ で $pP = pQ = Y$ なるものと、**Pred** の射 $(P \xrightarrow{u} Q)$ で $pu = \mathrm{id}_Y$ なるものについて、$\overline{f}_Q \circ f^*u = u \circ \overline{f}_P$ を満たすものと定められています。

$$\begin{tikzcd}
& f^*P \arrow[r, "\overline{f}_P"] \arrow[d, "f^*u"'] & P \arrow[d, "u"] \\
\mathbf{Pred} \arrow[d, "p"'] & f^*Q \arrow[r, "\overline{f}_Q"'] & Q \\
\mathbf{Set} & X \arrow[r, "f"'] & Y
\end{tikzcd}$$

7. ここで、$(-)^*$ や $\overline{(-)}$ は "関手的" (functorial)、つまり恒等射と合成に整合します。
$$\begin{aligned}
\mathrm{id}_Y^* Q &= Q, & (g \circ f)^*(Q) &= f^*(g^*Q), \\
\overline{\mathrm{id}_Y}(Q) &= \mathrm{id}_Q, & \overline{g \circ f}(Q) &= \overline{g}Q \circ \overline{f}(g^*Q).
\end{aligned}$$
8. 自己関手 $F : \mathbf{Set} \to \mathbf{Set}$ と、ファイブレーション $p : \mathbf{Pred} \to \mathbf{Set}$ について、
   関手 $\dot{F} : \mathbf{Pred} \to \mathbf{Pred}$ が、p に沿った F の「持ち上げ」(fibred lifting) であるとは、

$$\begin{tikzcd}
\mathbf{Pred} \arrow[r, "\dot{F}"] \arrow[d, "p"'] \arrow[dr, phantom, "//"] & \mathbf{Pred} \arrow[d, "p"] \\
\mathbf{Set} \arrow[r, "F"'] & \mathbf{Set}
\end{tikzcd}
\quad \text{かつ} \quad
\begin{tikzcd}
\mathbf{Pred}_Y \arrow[r, "f^*"] \arrow[d, "\dot{F}"'] \arrow[dr, phantom, "\circlearrowright"] & \mathbf{Pred}_X \arrow[d, "\dot{F}"] \\
\mathbf{Set}_{FY} \arrow[r, "(Ff)^*"'] & \mathbf{Set}_{FX}
\end{tikzcd}$$

を満たすことです。特に余代数との関わりの中では、以下の図のような状況を考えることが多いです。基底の関手 F をどうやって $\dot{F}$ に持ち上げるか（様相の入れ方、たとえば $\Box$ か $\Diamond$ か）が重要になります。(4 章で後述)

$$\begin{tikzcd}
\mathbf{Pred}_X \arrow[r, bend left, "\dot{F}_X"] & \mathbf{Pred}_{FX} \arrow[l, bend left, "c^*"]
\end{tikzcd}$$

$$\begin{tikzcd}
\dot{F}_X \circlearrowright \mathbf{Pred} \arrow[d, "p"] \\
F \circlearrowright \mathbf{Set}
\end{tikzcd}
\qquad
\begin{tikzcd}
P \arrow[r, mapsto] & \dot{F}_X P \arrow[dl, mapsto] \\
c^* \dot{F}_X P & \\
X \arrow[r, "c"'] & FX
\end{tikzcd}$$

以下、ファイブレーションの正確な定義を念のため書いておきます。（ですが、上記で扱った $p\colon \mathbf{Pred} \to \mathbf{Set}$ の考え方だけをきちんと丁寧に押さえておけば、次節以降の内容を理解するうえでそれほど支障はありません。）

定義 33（カルテシアン）　$p : \mathbb{E} \to \mathbb{B}$ を関手とします。$\mathbb{E}$ の射 $(X \xrightarrow{f} Y)$ が p-カルテシアン (p-cartesian) であるとは、すべての $Z \in \mathbb{E}$ について、以下のプルバック図式が可換になることである。

$$\begin{tikzcd}
\mathbb{E}(Z,X) \arrow[r, "f\circ(-)"] \arrow[d, "p_{ZX}"'] \arrow[dr, phantom, "\lrcorner", very near start] & \mathbb{E}(Z,Y) \arrow[d, "p_{ZY}"] \\
\mathbb{B}(pZ,pX) \arrow[r, "pf\circ(-)"'] & \mathbb{B}(pZ,pY)
\end{tikzcd}
\qquad \text{i.e.} \qquad
\begin{tikzcd}
\exists!\, h \arrow[r, mapsto] \arrow[d, mapsto] & \forall g \arrow[d, mapsto] \\
 & pg \\
\forall u \arrow[r, mapsto] & pf \circ u \arrow[u, equal]
\end{tikzcd}$$

注意　要するに以下と同じ。

$$\begin{tikzcd}
\mathbb{E} \arrow[d, "p"'] \\
\mathbb{B}
\end{tikzcd}
\qquad
\begin{tikzcd}
Z \arrow[r, dashed, "\exists!\, h"'] \arrow[rr, bend left, "\forall g"] & X \arrow[r, "f"'] & Y \\
pZ \arrow[r, "\forall u"'] \arrow[rr, bend left, "pg"] & pX \arrow[r, "pf"'] & pY
\end{tikzcd}$$

定義 34（ファイブレーション）　関手 $p : \mathbb{E} \to \mathbb{B}$ がファイブレーション (fibration) であるとは、$\mathbb{B}$ の任意の射 $u : I \to J$ と、J の上にある（つまり $pY = J$ であるような）任意の対象 $Y \in \mathbb{E}$ に対し、$\mathbb{E}$ の p-カルテシアン射 $f : X \to Y$ が u の上に存在する（つまり $pf = u$ である）ことである。

注意　要するに以下と同じ。

$$\begin{tikzcd}
\mathbb{E} \arrow[d, "p"'] \\
\mathbb{B}
\end{tikzcd}
\qquad
\begin{tikzcd}
 & Y \\
I \arrow[r, "u"'] & J
\end{tikzcd}
\quad \Longrightarrow \quad
\begin{tikzcd}
\exists X \arrow[r, "\text{cartesian}", "\exists f"'] & Y \\
I \arrow[r, "u"'] & J
\end{tikzcd}$$

定義 35（ファイバー）　$p : \mathbb{E} \to \mathbb{B}$ を関手とする。各対象 $I \in \mathbb{B}$ について、圏 $\mathbb{E}_I$ をファイバー (fiber) と呼ぶ。対象は、I の上にある（つまり $pX = I$ を満たすような）$X \in \mathbb{E}$ であり、射は id_I の上にある（つまり $pf = \mathrm{id}_I$ を満たすような）$\mathbb{E}$ の射 $f : X \to Y$ である。

定義 36（置換）　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとし、$u : I \to J$ を $\mathbb{B}$ の射とします。u の置換 (substitution) とは、関手 $u^* \colon \mathbb{E}_J \to \mathbb{E}_I$ であって以下を満たすものである。（ここで $\overline{u}_X \colon u^*X \to X$ および $\overline{u}_Y \colon u^*Y \to Y$ は、p-カルテシアン射でかつ $p(\overline{u}_X) = u$ および $p(\overline{u}_Y) = u$ を満たすものとする。）

$$\begin{tikzcd}
u^*X \arrow[r, "\overline{u}_X"] \arrow[d, dashed, "u^*f"'] & X \arrow[d, "f"] \\
u^*Y \arrow[r, "\overline{u}_Y"] & Y \\
 & \\
I \arrow[r, "u"] & J
\end{tikzcd}$$

- 対象 $X \in \mathbb{E}_J$ に対し、対象 $u^*X \in \mathbb{E}_I$ を返し、

- $\mathbb{E}_J$ の射 $f : X \to Y$ に対し、$\mathbb{E}_I$ の射 $u^*f : u^*X \to u^*Y$ を一意に定め、以下のプルバック図式を満たす:

$$\begin{tikzcd}
\mathbb{E}(u^*X, u^*Y) \arrow[r, "\overline{u}_Y \circ (-)"] \arrow[d, "p"'] \arrow[dr, phantom, "\lrcorner", very near start] & \mathbb{E}(u^*X, Y) \arrow[d, "p"] \\
\mathbb{B}(I, I) \arrow[r, "u \circ (-)"'] & \mathbb{B}(I, J)
\end{tikzcd}
\qquad \text{i.e.} \qquad
\begin{tikzcd}
\exists!\, u^*f \arrow[r, mapsto, "②"] \arrow[d, mapsto, "①"'] & f \circ \overline{u}_X \arrow[d, mapsto] \\
\mathrm{id}_I \arrow[r, mapsto] & u
\end{tikzcd}$$

すなわち、① $p(u^*f) = \mathrm{id}_I$ と ② $f \circ \overline{u}_X = \overline{u}_Y \circ u^*f$ を満たす。

注意 同一の射の上にあるカルテシアン射は ‘‘同型を除いて一意’’ (unique up to isomorphism) である。より正確に言うと、ファイブレーション $p : \mathbb{E} \to \mathbb{B}$ と、$\mathbb{B}$ の射 $u : I \to J$ が与えられ、$Y \in \mathbb{E}_J$ があり、2 つの p-カルテシアン射 $\overline{u}_Y : u^*Y \to Y$ と $f : X \to Y$ がいずれも u の上にあるとき、一意な同型射 $\alpha : X \xrightarrow{\cong} u^*Y$ がファイバー $\mathbb{E}_I$ に存在し、$f = \overline{u}_Y \circ \alpha$ が成り立つ。(証明略)

$$\begin{tikzcd}
 & X \arrow[r, "f"] \arrow[d, phantom, "\cong"] & Y \arrow[d, equal] \\
\mathbb{E} \arrow[d, "p"'] & u^*Y \arrow[r, "\overline{u}_Y"'] & Y \\
\mathbb{B} & I \arrow[r, "u"] & J
\end{tikzcd}$$

命題 37 $p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとする。$F : \mathbb{B} \to \mathbb{B}$, $\dot{F} : \mathbb{E} \to \mathbb{E}$ をそれぞれ $\mathbb{B}, \mathbb{E}$ の自己関手とし、$p \circ \dot{F} = F \circ p$ を満たすものとする。

$$\begin{tikzcd}
\mathbb{E} \arrow[r, "\dot{F}"] \arrow[d, "p"'] & \mathbb{E} \arrow[d, "p"] \\
\mathbb{B} \arrow[r, "F"'] & \mathbb{B}
\end{tikzcd}$$

このとき、$\mathbb{B}$ の任意の射 $(I \xrightarrow{u} J)$ に対し、以下の自然変換 $\gamma : \dot{F} \circ u^* \Rightarrow (Fu)^* \circ \dot{F}$ が存在する。

$$\begin{tikzcd}
\mathbb{E}_J \arrow[r, "u^*"] \arrow[d, "\dot{F}"'] \arrow[dr, phantom, "\swarrow \gamma"] & \mathbb{E}_I \arrow[d, "\dot{F}"] \\
\mathbb{E}_{FJ} \arrow[r, "(Fu)^*"'] & \mathbb{E}_{FI}
\end{tikzcd}
\qquad \text{i.e.} \qquad
\begin{tikzcd}
X \arrow[r, mapsto, "u^*"] \arrow[dd, mapsto] & u^*X \arrow[d, mapsto] \\
 & \dot{F}(u^*X) \arrow[d, "\gamma_X"] \\
\dot{F}X \arrow[r, mapsto] & (Fu)^*\dot{F}X
\end{tikzcd}$$

証明 $X \xmapsto{(pX = J)} J$ および $\dot{F}X \xmapsto{(p\dot{F}X = FpX = FJ)} FJ$ と考えると

$$\begin{tikzcd}
\dot{F} \circlearrowright \mathbb{E} \arrow[d, "p"] \\
F \circlearrowright \mathbb{B}
\end{tikzcd}
\qquad
\begin{tikzcd}
u^*X \arrow[r, "\overline{u}"] & X \\
I \arrow[r, "u"'] & J
\end{tikzcd}
\quad \longmapsto \quad
\begin{tikzcd}
\dot{F}(u^*X) \arrow[r, "\dot{F}\overline{u}"] & \dot{F}X \\
FI \arrow[r, "u"'] & FJ
\end{tikzcd}$$

が得られる。それゆえ、

$$\begin{tikzcd}
 & \dot{F}(u^*X) \arrow[d, dashed, "\gamma_X"'] \arrow[dr, bend left, "\dot{F}\overline{u}"] & \\
\mathbb{E} \arrow[d, "p"'] & (Fu)^*\dot{F}X \arrow[r, "\overline{Fu}"'] & \dot{F}X \\
\mathbb{B} & FI \arrow[r, "Fu"'] & FJ
\end{tikzcd}$$

となる。(γ の自然性の証明はここでは省略する。) □

定義 38（カルテシアンを保つ）　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとする。関手 $\dot{F} : \mathbb{E} \to \mathbb{E}$ が p-カルテシアンを保つ (preserves p-cartesian) とは、$\dot{F}$ が各 p-カルテシアン射 f を別の p-カルテシアン射 $\dot{F}f$ に送ること。

$$(X \xrightarrow{f} Y) \text{ in } \mathbb{E} \text{ is } p\text{-cartesian} \implies (\dot{F}X \xrightarrow{\dot{F}f} \dot{F}Y) \text{ in } \mathbb{E} \text{ is } p\text{-cartesian.}$$

命題 39　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとする。$F : \mathbb{B} \to \mathbb{B}$, $\dot{F} : \mathbb{E} \to \mathbb{E}$ をそれぞれ $\mathbb{B}, \mathbb{E}$ の自己関手とし、$p \circ \dot{F} = F \circ p$ を満たすものとする。（ここまでの状況は、先ほどの**命題 37** のときと同じ。）

$$\begin{tikzcd} \mathbb{E} \arrow[r, "\dot{F}"] \arrow[d, "p"'] & \mathbb{E} \arrow[d, "p"] \\ \mathbb{B} \arrow[r, "F"'] & \mathbb{B} \end{tikzcd}$$

このとき、以下の同値関係が成り立つ。

$$\dot{F} \text{ preserves } p\text{-cartesian} \iff \gamma : \dot{F} \circ u^* \Rightarrow (Fu)^* \circ \dot{F} \text{ is isomorphic.}$$

証明　任意の p-カルテシアン射 $f : X \to Y$ をとり、以下の状況を考える。

$$\begin{tikzcd} \mathbb{E}_{pY} \arrow[r, "(pf)^*"] \arrow[d, "\dot{F}"'] \arrow[dr, phantom, "\swarrow\gamma"] & \mathbb{E}_{pX} \arrow[d, "\dot{F}"] \\ \mathbb{E}_{FpY} \arrow[r, "(Fpf)^*"'] & \mathbb{E}_{FpX} \end{tikzcd} \qquad \text{i.e.} \qquad \begin{tikzcd} Y \arrow[r, mapsto] \arrow[dd, mapsto] & X \arrow[d, mapsto] \\ & \dot{F}X \arrow[d, "\gamma_Y"] \\ \dot{F}Y \arrow[r, mapsto] & (Fpf)^*\dot{F}Y \end{tikzcd}$$

言い換えると、状況は以下のとおり。

$$\begin{tikzcd} \mathbb{E} \arrow[d, "p"'] \\ \mathbb{B} \end{tikzcd} \qquad \begin{tikzcd} X \arrow[r, "f", "\overline{pf}"'] & Y \\ pX \arrow[r, "pf"'] & pY \end{tikzcd} \qquad \longmapsto \qquad \begin{tikzcd} \dot{F}X \arrow[d, dashed, "\gamma_Y"'] \arrow[dr, bend left, "\dot{F}f"] & \\ (Fpf)^*FY \arrow[r, "\overline{Fpf}"'] & \dot{F}Y \\ FpX \arrow[r, "Fpf"'] & FpY \end{tikzcd}$$

ここで、$\dot{F}f$ が p-カルテシアンである $\iff$ γ_Y が同型射である。（なぜなら、$\dot{F}f$ と $\overline{Fpf}$ はいずれも Fpf の上にあり、しかも $\overline{Fpf}$ は定義からして p-カルテシアンであるので、($\Rightarrow$): もし $\dot{F}f$ も p-カルテシアンなら、一意な同型射 γ_Y が手に入るし、($\Leftarrow$): もし γ_Y が同型射なら、$\dot{F}f$ は p-カルテシアンである。）　□

定義 40（関手の持ち上げ）　$p : \mathbb{E} \to \mathbb{B}$ をファイブレーションとする。$F : \mathbb{B} \to \mathbb{B}$ を $\mathbb{B}$ における自己関手とする。このとき、$\dot{F} : \mathbb{E} \to \mathbb{E}$ が p に沿った F のファイバー持ち上げ (fibered lifting) であるとは、$p \circ \dot{F} = F \circ p$ を満たし、かつ $\dot{F}$ がカルテシアンを保つ（cf. **定義 38**）ことである。

注意　すなわち、$\dot{F}$ は $\begin{tikzcd} \mathbb{E} \arrow[r, "\dot{F}"] \arrow[d, "p"'] \arrow[dr, phantom, "/\!/"] & \mathbb{E} \arrow[d, "p"] \\ \mathbb{B} \arrow[r, "F"'] & \mathbb{B} \end{tikzcd}$ と $\begin{tikzcd} \mathbb{E}_J \arrow[r, "u^*"] \arrow[d, "\dot{F}"'] \arrow[dr, phantom, "\cong"] & \mathbb{E}_I \arrow[d, "\dot{F}"] \\ \mathbb{E}_{FJ} \arrow[r, "(Fu)^*"'] & \mathbb{E}_{FI} \end{tikzcd}$ を満たす。

- 前者から、各 $X \in \mathbb{B}$ について、$\dot{F}_X : \mathbb{E}_X \to \mathbb{E}_{FX}$ を定義できることが従う。
- 後者は、「関手の持ち上げ $\dot{F}$ が引き戻し u^* と整合的である」ことを意味する。

さて、次章からは、本章で導入した道具立てをフル活用して、様々な種類の（非決定的、確率的、etc.）プログラムに対する最弱事前条件を定める方法を紹介します。

4　最弱事前条件をどのように手に入れるか ——— 圏論的な観点で

さて、本節では、非決定的プログラムや確率的プログラムなどといった様々な種類のプログラムに対する述語変換子意味論を得る方法について、圏論的な視座から考えます。一般に、状態の集合 X, Y と真理値の集合 Ω が与えられたときに、効果付き計算 $c : X \to FY$ に基づいて、事後条件 $q : Y \to \Omega$ を最弱事前条件 $wp[\![c]\!](q) : X \to \Omega$ に変換する関数である「述語変換子」 $wp[\![c]\!] : \Omega^Y \to \Omega^X$ を手に入れるためには、どんな手続きを経たらよいでしょうか。

まずはいくつか具体例を見てから、その後に一般論を考えましょう。

4.1　具体例を観察する

以下では、3 つのケースについて、それぞれ具体例を考えていきます。

具体例 (1)　決定的な 2 値的モデル検査の場合

- 真理値の集合を $\Omega = 2 = \{0, 1\}$ とします。
- 考えるべき完備束 $L = \Omega^X$ は、以下の通りです。（ここでは各 X を状態集合とします。）

$$\begin{aligned} &L = 2^X = \{q : X \to 2,\ \text{predicates}\} \\ &\ni \ \bot \text{ (everywhere false)}, \top \text{ (everywhere true)} \end{aligned}$$

- ここで、$f : X \to Y$ という計算を考えます。
  （なお、$Y = X$ の場合、以下のような余代数です。）

$$\left(\begin{array}{cccc} c: & X & \longrightarrow & X \\ & \Psi & & \Psi \\ & x & \mapsto & x' \end{array}\right) \qquad \textcircled{x_1} \longrightarrow \textcircled{x_2} \longrightarrow \cdots$$

- $f : X \to Y$ に対し、述語変換子 $\left(\begin{array}{cccc} f^*: & 2^Y & \longrightarrow & 2^X \\ & \Psi & & \Psi \\ & (X \xrightarrow{q} 2) & \mapsto & (X \xrightarrow{f^*(q)} 2) \end{array}\right)$ を

  $f^* : 2^Y \to 2^X$, $f^*(Q) = \{x \in X \mid f(x) \in Q\}$ で定めます。

- いま、以下の関数を定めましょう。

$$\begin{aligned} \exists_f : 2^X \to 2^Y,\quad \exists_f(P) &= \{f(x) \in Y \mid x \in P\} \\ &= \{y \in Y \mid \exists x \in X,\ f(x) = y \text{ and } x \in P\} \\ \forall_f : 2^X \to 2^Y,\quad \forall_f(P) &= \{y \in Y \mid f^*(\{y\}) \subseteq P\} \\ &= \{y \in Y \mid \forall x \in X,\ f(x) = y \text{ implies } x \in P\} \end{aligned}$$

- このとき、$\exists_f \dashv f^* \dashv \forall_f$ が成り立ちます。

$$\begin{tikzcd} 2^X \arrow[r, bend left=60, "\exists_f"] \arrow[r, bend right=60, "\forall_f"'] & 2^Y \arrow[l, "\bot"] \arrow[l, "\bot"'] \end{tikzcd}$$

$$\frac{\exists_f(P) \sqsubseteq Q \text{ in } 2^Y}{P \sqsubseteq f^*(Q) \text{ in } 2^X} \quad \text{and} \quad \frac{f^*(Q) \sqsubseteq P \text{ in } 2^X}{Q \sqsubseteq \forall_f(P) \text{ in } 2^Y}$$

具体例 (2)　非決定的な 2 値的モデル検査の場合

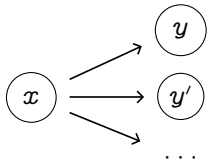
- 真理値の集合を $\Omega = 2 = \{0, 1\}$ とします。
- 考えるべき完備束 $L = \Omega^X$ は、以下の通りです。

$$
\begin{aligned}
&L = 2^X = \{q : X \to 2,\ \text{predicates}\} \\
&\ni \ \bot\,(\text{everywhere false}),\ \top\,(\text{everywhere true})
\end{aligned}
$$

（ここまでは一つ前の例と同じです。）

- ここで、$f : X \to \mathcal{P}Y$ という計算を考えます。
（なお、$Y = X$ の場合、以下のような余代数です。）

$$
\left(\begin{array}{lccccl}
c: & X & \longrightarrow & \mathcal{P}X & & \\
 & \Psi & & \Psi & & \\
 & x & \mapsto & c(x) & = & \{x' \mid x \to x'\}
\end{array}\ \text{Kripke structure}\right)
$$



- さて、述語変換子 $\left(\begin{array}{lccc} f^*: & 2^Y & \longrightarrow & 2^X \\ & \Psi & & \Psi \\ & (X \xrightarrow{q} 2) & \longmapsto & (X \xrightarrow{f^*(q)} 2) \end{array}\right)$ をどうやって手に入れればよいでしょうか？

以下のようにして得られます。（上段が入力、下段が出力です。）

$$
\frac{q : Y \to 2,\ \text{post-condition}}{\left(\begin{array}{lccccccc}
f^*(q): & X & \xrightarrow{f} & \mathcal{P}Y & \xrightarrow{\mathcal{P}q} & \mathcal{P}2 & \xrightarrow[\text{modality}]{\tau} & 2 \\
 & \Psi & & \Psi & & \Psi & & \Psi \\
 & x & \longmapsto & \{x_1, \ldots, x_n\} & \longmapsto & \{q(x_1), \ldots, q(x_n)\} & \longmapsto & 0 \text{ or } 1
\end{array}\right)}
$$

- $f : X \to \mathcal{P}Y$ に対し、
述語変換子 f^* は、様相 $\tau : \mathcal{P}2 \to 2$（パラメーターマップとも呼ばれる）の選び方に拠って決まります。
f^* を以下のように 2 通りの異なるしかたで定めることができます。
$\Box_f(-) : 2^Y \to 2^X,\ q \mapsto \{x \in X \mid \forall y \in Y\,(x \to y \text{ implies } y \in q)\}$
$\Diamond_f(-) : 2^Y \to 2^X,\ q \mapsto \{x \in X \mid \exists y \in Y\,(x \to y \text{ and } y \in q)\}$

- いま、以下の関数を定めましょう。
$\exists_f : 2^X \to 2^Y,\ p \mapsto \{y \in Y \mid \exists x \in X,\ x \to y \text{ and } x \in p\}$
$\forall_f : 2^X \to 2^Y,\ p \mapsto \{y \in Y \mid \forall x \in X,\ x \to y \text{ implies } x \in p\}$

- このとき、$\exists_f \dashv \Box_f$ および $\Diamond_f \dashv \forall_f$ がともに成り立ちます。

$$
\begin{tikzcd}
2^X \arrow[r, bend left, "\exists_f"] \arrow[r, phantom, "\perp"] & 2^Y \arrow[l, bend left, "\Box_f"]
\end{tikzcd}
\qquad\qquad
\begin{tikzcd}
2^X \arrow[r, bend right, "\forall_f"'] \arrow[r, phantom, "\perp"] & 2^Y \arrow[l, bend right, "\Diamond_f"']
\end{tikzcd}
$$

$$
\frac{\exists_f(P) \sqsubseteq Q \text{ in } 2^Y}{\overline{P \sqsubseteq \Box_f(Q) \text{ in } 2^X}}
\quad \text{and} \quad
\frac{\Diamond_f(Q) \sqsubseteq P \text{ in } 2^X}{\overline{Q \sqsubseteq \forall_f(P) \text{ in } 2^Y}}
$$

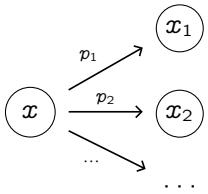
具体例 (3) 確率的モデル検査の場合

- 真理値の集合を $\Omega = [0,1]$ とします。
- 考えるべき完備束 $L = \Omega^X$ は、以下の通りです。

$$L = [0,1]^X = \{q : X \to [0,1],\ \text{predicates as random variables}\}$$

- ここで、$f : X \to \mathcal{D}Y$ という計算を考えます。
  （なお、$Y = X$ の場合、以下のような余代数です。）

$$\Big(c : X \to \mathcal{D}X = \{d : X \to [0,1],\ X \text{ 上の確率分布}\}\Big)$$



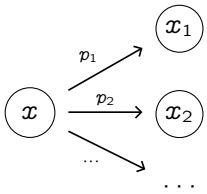
- さて、述語変換子 $f^* : [0,1]^Y \to [0,1]^X$ をどうやって手に入れればよいでしょうか？
  以下のようにして得られます。（上段が入力、下段が出力です。）

$$\frac{q : Y \to [0,1],\ \ \text{post-condition}}{\left(\begin{array}{cccccccc} f^*(q) : & X & \xrightarrow{f} & \mathcal{D}Y & \xrightarrow{\mathcal{D}q} & \mathcal{D}[0,1] & \xrightarrow[=Av]{\tau} & [0,1] \\ & \Psi & & \Psi & & \Psi & & \Psi \\ & x & \longmapsto & \sum p_i|x_i\rangle & \longmapsto & \sum p_i|q(x_i)\rangle & \longmapsto & \sum p_i \cdot q(x_i) \end{array}\right)}$$

- $f : X \to \mathcal{P}Y$ に対し、述語変換子 $f^* : [0,1]^Y \to [0,1]^X$ は以下で定義されます。

$$f^*(q)(x) = \sum_{x' \in X} f(x)(x') \cdot q(x')$$

  つまり、$f^*(q)$ は、各 x に対し、q による値の期待値 $p_1 \cdot q(x_1) + p_2 \cdot q(x_2) + \cdots$ を割り当てます。



- なお、この f^* は、左右いずれの随伴も持ちません。
  （随伴がないと不便な場面がいろいろあるのですが、対応策としては、米田埋め込みを使う方法があります。詳細は省略しますが、ざっくりとしたアイデアとしては、おおむね下の図の通りです。やや複雑な話なので、読み飛ばして構いません。）

$$\begin{tikzcd} \widehat{L} \arrow[dr, shift left, "\mathrm{Ran}_h(y)"] \arrow[dr, phantom, "\top"] & \\ L \arrow[u, "y"] \arrow[r, "h"'] & L' \arrow[ul, shift left, "\mathrm{Lan}_y(h)"] \end{tikzcd} \qquad \begin{tikzcd} \widehat{L} \arrow[r, shift right, "\mathrm{Lan}_y(y \circ f^*)"'] & \widehat{L} \arrow[l, shift right, "\exists g"'] \arrow[ul, phantom, "\top"] \\ L \arrow[u, "y"] \arrow[r, "f^*"'] & L \arrow[u, "y"'] \end{tikzcd}$$

$$\mathrm{Lan}_y(h)(P) = \bigsqcup_{x \in P} h(x), \qquad \mathrm{Lan}_y(y \circ f^*)(P) = \{z \in L \mid \exists x \in P,\ z \sqsubseteq f^*(x)\}$$

4.2 一般論を展開する

これまで見た具体例をふまえて、一般論を展開していきましょう。

- 副作用つき計算 $c : X \to FY$、真理値の集合 Ω、様相 $\tau : F\Omega \to \Omega$ が与えられたとき、述語変換子 $wp[\![c]\!] : \Omega^Y \to \Omega^X$ は以下のしかたで得られます。

$$\frac{Y \xrightarrow{q} \Omega, \ \text{post-condition}}{wp[\![c]\!](q) : X \xrightarrow{c} FY \xrightarrow{Fq} F\Omega \xrightarrow{\tau} \Omega, \ \text{weakest pre-condition}}$$

  このように $wp[\![c]\!](q) = \tau \circ Fq \circ c$ で定まります。

これを、グロタンディーク・ファイブレーションの言葉で次のように捉えることができます。

- （状況設定） ファイブレーション $p : \mathbf{Pred} \to \mathbf{Set}$ を考えます。各ファイバー $\mathbf{Pred}_X = \Omega^X$ は「各状態集合 X 上の述語全体」であり順序集合をなし、各 $f : X \to Y$ に対する置換 $f^* : \Omega^Y \to \Omega^X$, $q \mapsto q \circ f$ は「事後条件 q を f によって引き戻した事前条件」$f^*(q)$ を返します。

- （課題） いま、副作用つき計算 $c : X \to FY$ が与えられたとき、単に c^* を考えると、これは $\Omega^{FX} \to \Omega^X$ という型を持ちます。しかし、われわれが欲しい述語変換子は、$\Omega^Y \to \Omega^X$ という型を持つものです。
  （というのも、事後条件（Y 上の述語）から事前条件（X 上の述語）への変換を行いたいからです。）

- （解決策） ここで、様相 $\tau : F\Omega \to \Omega$ が重要な役割を果たします。この τ は、「Y 上の述語」から「FY 上の述語」へと持ち上げる操作、**述語持ち上げ** (predicate lifting) を誘導します。

  具体的には、各 Y について、

$$\dot{F}_Y : \Omega^Y \to \Omega^{FY}, \quad \dot{F}_Y(q) := \tau \circ Fq \qquad (\text{ただし } q : Y \to \Omega \text{ は任意の述語とする})$$

  と定めます。この $\dot{F}_Y$ は、「Y 上の述語 q を、計算効果 F で包まれた状態空間 FY 上の述語 $\dot{F}_Y(q)$ へと持ち上げる」操作です。これを、**述語持ち上げ**と呼びます。（通常、τ には単調性を課し、$\dot{F}_Y$ が単調になるようにします。）

  すると、最弱事前条件 $wp[\![c]\!]$ は、この「述語持ち上げ $\dot{F}_Y$」と「計算 c の引き戻し c^*」の合成として書けます。

$$\begin{tikzcd} \Omega^Y \arrow[r, "\dot{F}_Y"] \arrow[dr, "wp[\![c]\!]"'] & \Omega^{FY} \arrow[d, "c^*"] \\ & \Omega^X \end{tikzcd}$$

  すなわち、任意の事後条件 $q : Y \to \Omega$ に対して、以下のように理解できます。

$$\begin{aligned} wp[\![c]\!](q) &= c^*(\dot{F}_Y(q)) \\ &= c^*(\tau \circ Fq) \\ &= \tau \circ Fq \circ c \end{aligned}$$

まとめると、述語変換子は

1) Y 上の述語 q を、様相 τ を用いて FY 上に**持ち上げて**、
2) それを 計算 c に沿って X 上に**引き戻す**、

という 2 段階のプロセスとして理解できます。（スローガン:「**持ち上げて、引き戻す！**」）

なお余代数 $c : X \to FX$ を考える場合は、まさに以下の図のようになります。

$$
\begin{tikzcd}
\mathbf{Pred} \arrow[loop left, "\dot{F}_X"] \arrow[d, "p"] \\
\mathbf{Set} \arrow[loop left, "F"]
\end{tikzcd}
\qquad
\begin{tikzcd}
\Omega^X \arrow[r, bend left, "\dot{F}_X"] & \Omega^{FX} \arrow[l, bend left, "c^*"] \\
q \arrow[r, mapsto] & \dot{F}_X q \arrow[l, mapsto] \\
c^* \dot{F}_X q & \\
X \arrow[r, "c"'] & FX
\end{tikzcd}
$$

4.3 応用例: 確率的プログラムの「停止性」および「期待実行時間」の検証

最後に、「最弱事前条件意味論の圏論的定式化が実際にどのようにモデル検査に役立つか」を紹介するという目的で、確率的プログラムのモデル検査に関して、筆者が最近行った研究の内容を（少しだけ）取り上げます。

まだ

5　まとめ

参考文献

[1] 蓮尾 (2011) 「Hoare 論理によるプログラム検証」(資料), [URL].

[2] 郡 (2025)「圏で圏を集める？—ファイブレーション入門」, 『圏論の歩き方 [改訂版]』, 日本評論社.

[3] Adámek J, Milius S, Moss LS. *Initial Algebras and Terminal Coalgebras: The Theory of Fixed Points of Functors*. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press.

[4] Beck, J. (1969) Distributive laws. In: Seminar on Triples and Categorical Homology Theory (Zürich, 1966/1967), vol. 80, pp. 119-140. Lecture Notes in Mathematics. Springer.

[5] Jacobs, B. (1999) *Categorical logic and type theory*, Elsevier.

[6] Jacobs, B. (2016) *Introduction to coalgebra: towards mathematics of states and observation*, Cambridge University Press.

[7] Hasuo, I. (2015) Generic weakest precondition semantics from monads enriched with order, TCS.

[8] Hasuo, I. (2025) *Abstract and concrete model checking*, lecture notes for Marktoberdorf summer school 2025, [URL].

[9] nLab authors. (2023) *List of notable initial algebras and terminal coalgebras*, [URL].